

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**Autonomous UAV-Based Emergency Medical Delivery System in
GPS-Denied and Communication-Degraded Environments**

Submitted by

Tran Thi Ngoc Huyen – ITITIU23009

Date Submitted: [04/09/2026]

Date Performed: [04/09/2026]

Course Instructor: Le Thanh Lanh

Table of Contents

CHAPTER 1: INTRODUCTION	7
1.1 Background and Motivation	7
1.2 Problem Statement	8
1.2.1 Reliable Localization in GPS-Denied Conditions	8
1.2.2 Safe Transition Between Localization Sources	8
1.2.3 Accurate Landing at the Target Location	8
1.2.4 Communication During Network Degradation	8
1.3 Research Objectives	8
1.4 Scope of the Pre-Thesis	9
1.5 Expected Contributions	10
CHAPTER 2: RELATED WORK AND TECHNOLOGIES	11
2.1 UAV-Based Emergency Medical Delivery	11
2.2 GPS-Denied Navigation	11
2.2.1 Global Positioning System Limitations	11
2.2.2 Visual-Inertial Odometry	11
2.2.3 Visual SLAM	12
2.3 ORB-SLAM3	12
2.3.1 Overview	12
2.3.2 Tracking and Mapping	12
2.3.3 Selection of ORB-SLAM3	13
2.4 Camera Calibration for Visual Localization	13
2.5 Precision Landing Using Fiducial Markers	13
2.5.1 Need for Precision Landing	13
2.5.2 Fiducial Markers	13
2.5.3 ArUco Marker	14
2.6 PX4 EKF2 and Multi-Source State Estimation	14
2.6.1 Sensor Fusion in Autonomous UAVs	14
2.6.2 PX4 EKF2	14

2.6.3 GPS and VIO Complementarity	15
2.7 LoRa for Backup Communication	15
2.7.1 Communication Requirements	15
2.7.2 LoRa Communication	15
2.7.3 Backup Communication Architecture	15
2.8 Obstacle Detection	16
2.8.1 Motivation	16
2.8.2 YOLO-Based Object Detection.....	16
2.8.3 Monocular Depth Estimation	16
2.8.4 Current Status	16
2.9 Summary of Related Technologies	17
CHAPTER 3: SYSTEM ARCHITECTURE	18
3.1 Overall System Architecture	18
3.2 Ground-Side Software Architecture.....	19
3.3 Companion-Computer Software Architecture	19
3.4 Camera Sharing Architecture	20
3.5 ROS2 Communication Architecture	21
3.6 Communication Between ROS2 and PX4	22
3.7 Localization Architecture.....	23
3.8 Precision Landing Architecture.....	23
3.9 Development and Simulation Environment	25
3.10 System Design Considerations.....	26
CHAPTER 4: GPS-DENIED LOCALIZATION USING VIO/SLAM.....	28
4.1 ORB-SLAM3 Fundamentals.....	28
4.1.1 Tracking.....	28
4.1.2 Mapping.....	28
4.1.3 Loop Closing	29
4.2 ORB-SLAM3 Deployment and Implementation	29
4.3 Camera Calibration	30

4.3.1 Calibration Procedure	30
4.3.2 Reprojection Error	30
4.4 ROS2 Integration	31
4.4.1 Shared Camera Input	32
4.5 Localization Results	32
4.6 Pseudo-Dense Reconstruction Using Depth Anything V2	33
4.7 Discussion	34
CHAPTER 5: ADAPTIVE GPS-to-VIO LOCALIZATION SWITCHING	35
5.1 Motivation for Conditional Switching	35
5.2 Quantitative Switching Conditions	35
5.3 Hysteresis for Stable Switching	36
5.4 Integration with PX4 EKF2	37
5.5 SITL Testing and Results	38
5.6 Limitations	39
CHAPTER 6: PRECISION LANDING USING ARUCO MARKER	40
6.1 Precision Landing Method	40
6.2 Landing Control State Machine	40
6.3 Safety Mechanism	41
6.4 Handoff from Navigation to Precision Landing	41
6.5 Experimental Results	42
6.6 Limitations	43
CHAPTER 7: BACKUP COMMUNICATION USING LoRa	44
7.1 Role of LoRa as a Backup Communication Channel	44
7.2 Lightweight Binary Packet Design	44
7.3 Transport and Protocol Architecture	44
7.4 Automatic Fallback Mechanism	45
7.5 Testing and Results	45
7.6 Limitations	46
CHAPTER 8: OBSTACLE DETECTION PROTOTYPE	47

8.1 Overview	47
8.2 YOLO-Based Object Detection	47
8.3 Depth Estimation Using Depth Anything V2	47
8.4 Combined Perception Pipeline	47
8.5 Prototype Evaluation	48
8.6 Limitations	49
CHAPTER 9: SYSTEM INTEGRATION AND TESTING	50
9.1 Testing Methodology	50
9.2 Scenario A – Precision Landing End-to-End	50
9.3 Scenario B – GPS-to-VIO Switching in SITL	53
9.4 Scenario C – Automatic LoRa Fallback.....	54
9.5 Testing Environment and Tools	55
CHAPTER 10: LIMITATIONS AND DISCUSSION	56
10.1 Development Environment Limitations	56
10.2 Limitations of Simulation-Based Testing	56
10.3 Technical Limitations of Individual Modules	56
10.4 High-Level Control Arbitration	57
CHAPTER 11: FUTURE WORK AND THESIS PLAN	58
11.1 Hardware Deployment	58
11.2 Real Camera Calibration and Integration.....	58
11.3 Integration with Real Flight Hardware	58
11.4 Real-World Flight Testing	59
11.5 Physical LoRa Implementation	59
11.6 High-Level Control Arbitration	59
CHAPTER 12: CONCLUSION	61
12.1 Summary of Achieved Results	61
12.2 Conclusion and Feasibility	61
REFERENCES	62

Table of Figures

Figure 3.1: Overall System Architecture	18
Figure 3.2: Ground-side Communication Architecture	19
Figure 3.3: ROS2 Node Architecture on Companion Computer	20
Figure 3.4: Camera Sharing Architecture	21
Figure 3.5: Localization Architecture	23
Figure 4.2: Camera Calibration Result	31
Figure 6.3: Precision Landing Completion and Successful Touchdown	43
Figure 8.1: YOLO and Depth Anything V2-Based Obstacle Detection Pipeline.....	48
Figure 8.2: YOLO-Based Object Detection with Relative Depth Estimation	49
Figure 9.1: Emergency Request and Drone Dispatch Interface.....	50
Figure 9.2: UAV Navigation from the Initial Position to the Target Area in QGroundControl	51
Figure 9.3: Precision Landing Controller Execution and PX4 AUTO.LAND Handoff.....	51
Figure 9.4: Precision Landing Status Transition and Successful Landing	52
Figure 9.5: PX4 Flight Controller Log Confirming Takeoff, Landing, and Disarming	52
Figure 9.6: Completed Emergency Request	53
Figure 9.7: GPS-to-VIO-to-GPS Localization Source Switching	54
Figure 9.8: Detection of Primary Network Failure and Activation of LoRa Fallback	54
Figure 9.9: Detection of Primary Network Failure and Switching to LoRa Fallback	55
Figure 9.10: Primary Network Reconnection After LoRa Fallback	55

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Unmanned Aerial Vehicles (UAVs) have become increasingly useful in applications that require rapid transportation, remote operation, and autonomous navigation. One promising application is the delivery of emergency medical supplies to locations where conventional transportation may be slow, difficult, or unavailable. In emergency situations such as natural disasters, isolated areas, or infrastructure disruptions, UAVs can potentially provide a faster and more flexible method for transporting essential medical resources.

However, autonomous UAV operation in emergency environments presents several significant challenges. Conventional UAV navigation relies heavily on the Global Positioning System (GPS) or other Global Navigation Satellite Systems (GNSS) for position estimation. In real-world environments, GPS signals may become unavailable or unreliable due to signal obstruction, interference, urban structures, or other environmental conditions. A UAV that depends exclusively on GPS may therefore experience degraded navigation performance or lose its ability to maintain an accurate position when GPS information becomes unavailable.

Another important challenge is the ability to accurately reach and land at the intended destination. Position estimates obtained from GPS are generally not sufficient for applications requiring highly accurate landing at a specific location. For emergency medical delivery, inaccurate landing may cause the payload to be delivered away from the intended target and may also introduce safety risks. Therefore, a visual-based precision landing mechanism is required to guide the UAV during the final landing phase.

Communication reliability is also an important consideration. The proposed system uses an Internet-based communication channel for interaction between the ground-side application and the UAV software system. However, emergency environments may involve unstable or unavailable network connectivity. Consequently, a backup communication mechanism is required to maintain essential communication when the primary network connection is degraded.

Based on these challenges, this project investigates an autonomous UAV-based emergency medical delivery system capable of operating under GPS-degraded or GPS-denied conditions while maintaining a reliable navigation and communication pipeline. The proposed system combines visual-inertial localization, adaptive localization-source switching, vision-based precision landing, and a backup communication mechanism into a unified software architecture.

For the pre-thesis stage, the proposed system is developed and evaluated primarily in a simulation environment. PX4 Software-In-The-Loop (SITL) is used to simulate the UAV flight controller and vehicle behavior, while a real webcam and camera calibration procedure are used to investigate the vision-based components. Real hardware flight testing is outside the scope of the current pre-thesis stage and is planned for the subsequent thesis phase.

1.2 Problem Statement

The main problem addressed in this project is the development of a software architecture for autonomous UAV emergency medical delivery when conventional GPS navigation and primary communication channels cannot be fully relied upon.

The problem can be divided into four main requirements.

1.2.1 Reliable Localization in GPS-Denied Conditions

The UAV should be capable of obtaining position information even when GPS is unavailable or unreliable. Visual-Inertial Odometry (VIO) and Visual SLAM techniques can provide an alternative source of localization by using visual information from a camera together with motion-related information.

1.2.2 Safe Transition Between Localization Sources

Simply switching between GPS and VIO whenever GPS becomes unavailable may result in unstable behavior, particularly when the GPS signal is close to its reliability threshold. Therefore, the system requires a condition-based switching mechanism that evaluates the quality and availability of localization sources before changing the active source.

The proposed approach considers factors such as GPS fix status, position covariance, and the freshness of VIO data. A hysteresis mechanism is also considered to prevent repeated switching when the system operates near a decision threshold.

1.2.3 Accurate Landing at the Target Location

Even if the UAV can navigate to the general destination, accurate delivery requires a more precise landing mechanism. The system therefore uses a visual fiducial marker, specifically an ArUco marker, to identify the landing target and estimate the relative position of the UAV with respect to the marker.

The landing process is organized as a state-based procedure including searching for the marker, aligning with the target, descending, and finally handing control to the appropriate landing mechanism.

1.2.4 Communication During Network Degradation

The primary communication channel of the system relies on a network-based connection. If this connection becomes unavailable, the UAV should have an alternative communication path for essential commands or status information. LoRa is therefore investigated as a low-bandwidth backup communication mechanism.

The system includes a mechanism for detecting the state of the primary WebSocket connection and switching to the backup communication path when necessary.

1.3 Research Objectives

The primary objective of this project is to design, implement, and evaluate a software architecture for an autonomous UAV emergency medical delivery system operating in GPS-degraded and communication-degraded environments.

The specific objectives of the pre-thesis stage are as follows:

1. To investigate a vision-based localization approach using ORB-SLAM3 for GPS-denied navigation.
2. To calibrate the camera and evaluate its suitability for the visual localization pipeline.
3. To integrate the visual localization system into a ROS2-based software architecture.
4. To develop a condition-based mechanism for switching between GPS and VIO localization sources.
5. To use the existing PX4 EKF2 sensor-fusion mechanism instead of implementing a separate Kalman Filter.
6. To develop a vision-based precision landing mechanism using an ArUco marker.
7. To design a state machine for the precision landing process and introduce safety mechanisms for temporary marker loss.
8. To investigate LoRa as a backup communication channel when the primary network connection becomes unavailable.
9. To evaluate the proposed modules through controlled simulation scenarios using PX4 SITL and related software tools.
10. To establish a software foundation that can later be transferred to real UAV hardware during the full thesis stage.

The central technical focus of the pre-thesis is the adaptive transition between GPS and VIO. Rather than treating GPS and VIO as completely independent navigation systems, the project investigates how the system can determine when one source should be preferred over another and how this transition can be performed more reliably.

1.4 Scope of the Pre-Thesis

The scope of this pre-thesis is limited to software development, integration, and simulation-based validation. The complete system is not yet deployed on a real UAV for flight testing.

The current implementation focuses on four main functional modules:

- GPS-denied localization using VIO/SLAM;
- adaptive GPS-to-VIO and VIO-to-GPS localization switching;
- vision-based precision landing using an ArUco marker; and
- backup communication using a LoRa-based architecture.

In addition, an obstacle detection prototype using YOLO and Depth Anything V2 is investigated as a supplementary feature. This component is currently at the prototype stage and has not yet been integrated as a complete ROS2 node within the main flight-control pipeline.

The experimental environment consists primarily of PX4 SITL, a simulation environment such as Gazebo or jMAVSim, ROS2, MAVROS2, and a real webcam for vision-related experiments. Camera calibration is also performed using real camera data.

The following aspects are explicitly outside the current pre-thesis scope:

- Real UAV flight testing;
- Deployment on Jetson Nano;
- Integration with a physical Pixhawk flight controller for flight operation;
- Integration with the SIYI MK15 and Air Unit during real flight;

- Physical LoRa range and radio-performance testing;
- Evaluation under real GPS interference or real-world GPS noise; and
- Complete real-world obstacle avoidance.

These activities are planned for the full thesis phase.

1.5 Expected Contributions

The main expected contribution of this project is a condition-based GPS-to-VIO localization switching mechanism designed for autonomous UAV operation in environments where GPS reliability may change over time.

Instead of performing a simple binary decision based only on GPS availability, the proposed mechanism considers multiple indicators of localization quality, including GPS fix status, position covariance, and the freshness of VIO measurements. Hysteresis is incorporated to reduce unstable switching when the system operates close to predefined thresholds.

Another contribution is the integration of this localization strategy into a ROS2-based UAV software architecture while utilizing the sensor-fusion capabilities already provided by PX4 EKF2. This approach avoids unnecessarily implementing an independent Kalman Filter and allows the flight controller to perform the required state estimation and fusion.

The project also contributes an end-to-end software architecture combining navigation, precision landing, and communication components. The architecture connects a ground-side web application to the UAV through a relay server, bridge node, MAVROS2, and PX4. This provides a foundation for further hardware-based implementation in the full thesis phase.

Finally, the project provides simulation-based experimental evidence for the feasibility of the proposed architecture. The testing focuses on isolated scenarios, including precision landing, GPS-to-VIO switching, and automatic communication fallback, allowing the behavior of individual modules to be evaluated before attempting full system integration.

CHAPTER 2: RELATED WORK AND TECHNOLOGIES

2.1 UAV-Based Emergency Medical Delivery

Unmanned Aerial Vehicles (UAVs) have been investigated for a variety of transportation and logistics applications because of their ability to operate without conventional road infrastructure and to reach destinations relatively quickly. One particularly important application is emergency medical delivery, where UAVs can be used to transport medical supplies to locations that may be difficult to access using conventional transportation.

An autonomous medical delivery UAV must satisfy several requirements beyond basic flight control. The system must be able to determine its position, navigate toward the destination, communicate with external operators or systems, and perform a safe and accurate landing. These requirements become more challenging in emergency environments because GPS availability, communication infrastructure, and environmental conditions cannot always be guaranteed.

For this reason, the system investigated in this project focuses on combining multiple technologies rather than relying on a single navigation or communication method. Visual localization is used as an alternative source of position information, a condition-based switching mechanism is used to manage GPS and VIO, visual markers are used for precision landing, and LoRa is considered as a backup communication channel.

2.2 GPS-Denied Navigation

2.2.1 Global Positioning System Limitations

GPS is commonly used by autonomous UAVs to estimate their global position. Under normal conditions, GPS provides useful information for navigation and waypoint-based flight. However, GPS performance can degrade in environments where satellite signals are obstructed, interfered with, or otherwise unreliable.

Examples include dense urban environments, indoor or partially covered environments, disaster areas, and locations where intentional or unintentional interference affects satellite-based positioning.

A navigation system that depends exclusively on GPS may therefore become unreliable when the GPS signal is lost. For autonomous UAV applications, this creates a need for alternative localization methods that can continue operating without direct dependence on satellite positioning.

2.2.2 Visual-Inertial Odometry

Visual-Inertial Odometry (VIO) estimates the motion of a platform using visual observations and inertial measurements. A camera provides information about changes in the visual environment, while inertial measurements provide information about motion dynamics. By combining these sources, a system can estimate the relative movement of the UAV over time.

Compared with GPS, VIO does not directly provide an absolute global position. Instead, it generally estimates motion relative to a reference frame established during operation. This makes VIO particularly useful as a complementary localization source when GPS becomes unavailable.

However, VIO performance depends strongly on the quality of visual observations, camera calibration, environmental texture, lighting conditions, and the quality of the available sensor data. Consequently, VIO should not simply be considered a universal replacement for GPS. Instead, it is more appropriate to treat GPS and VIO as complementary sources whose reliability depends on the operating conditions.

2.2.3 Visual SLAM

Visual Simultaneous Localization and Mapping (Visual SLAM) extends the concept of visual motion estimation by simultaneously estimating the camera trajectory and constructing a representation of the surrounding environment.

A typical Visual SLAM system consists of several major processes, including visual feature extraction and tracking, camera pose estimation, map construction, and loop-closure detection. The resulting map can provide additional information that helps maintain localization over an extended trajectory.

In the proposed system, Visual SLAM is investigated as the foundation for GPS-denied localization. Among the available approaches, ORB-SLAM3 is selected for the initial implementation.

2.3 ORB-SLAM3

2.3.1 Overview

ORB-SLAM3 is a Visual SLAM system designed to support different sensor configurations and visual-inertial estimation. It provides mechanisms for camera tracking, map construction, and loop closing, making it suitable for investigating camera-based localization.

The system is particularly relevant to this project because the objective is not only to detect visual features but also to maintain a continuously estimated camera trajectory in an environment where GPS may not be available.

2.3.2 Tracking and Mapping

The tracking component processes incoming camera observations and attempts to determine the current camera pose relative to the existing map. Visual features are used to establish correspondences between consecutive observations and previously observed locations.

The mapping component maintains the spatial representation required for continued localization. Keyframes and map points provide the structure through which the system can relate new observations to previously observed areas.

Loop closing provides an additional mechanism for correcting accumulated trajectory errors when the system recognizes a previously visited area. This is important because purely incremental visual estimation can accumulate errors over time.

2.3.3 Selection of ORB-SLAM3

ORB-SLAM3 was selected for the pre-thesis implementation because it provides an established Visual SLAM framework that can be integrated into the proposed ROS2 architecture. Rather than developing a complete SLAM algorithm from the beginning, the project focuses on integrating an existing localization framework into a UAV software pipeline.

The implementation includes building ORB-SLAM3 from source, resolving software dependencies such as Pangolin, and adapting the system to the required C++ environment. The resulting system is then connected to ROS2 through a dedicated VIO node.

2.4 Camera Calibration for Visual Localization

Camera calibration is an important component of a vision-based localization system. The camera model determines how three-dimensional points in the environment are projected onto the image plane. Errors in the intrinsic parameters or distortion model can therefore affect subsequent pose estimation.

The calibration process estimates parameters such as focal length, principal point, and lens distortion coefficients. A calibration target is observed from multiple orientations, allowing the system to estimate the camera model.

The quality of the calibration can be evaluated using reprojection error. A lower reprojection error generally indicates that the estimated camera model can reproduce the observed calibration points more accurately.

In this project, a real webcam is calibrated before being used in the visual processing pipeline. The calibration results are subsequently used by the VIO/SLAM component. This allows the project to investigate the effect of a real camera rather than relying exclusively on simulated camera parameters. Camera calibration and its influence on localization accuracy are therefore included as part of the experimental evaluation.

2.5 Precision Landing Using Fiducial Markers

2.5.1 Need for Precision Landing

Navigation to a destination and accurate landing are two different problems. A UAV may successfully reach the general area of a target while still having a significant horizontal position error during the final landing phase.

For emergency medical delivery, the landing position is important because the payload must be delivered to the intended location. A vision-based landing system can therefore provide a more precise reference during the final stage of the flight.

2.5.2 Fiducial Markers

Fiducial markers are artificial visual patterns that can be detected by a camera and used to estimate their position relative to the camera. They provide a structured visual reference that is easier to detect and identify than arbitrary environmental features.

Common fiducial marker systems include ArUco and AprilTag. These approaches can provide both marker identification and geometric information that can be used for relative pose estimation.

2.5.3 ArUco Marker

The proposed system uses an ArUco marker as the visual landing target. Once the marker is detected, the system can estimate its relative pose using the known geometry of the marker and the calibrated camera parameters.

The relative position between the UAV camera and the marker can then be used to determine the horizontal alignment error. This error provides the basis for generating appropriate control commands during the precision landing process.

ArUco is selected for this project because it provides a relatively straightforward approach for marker detection and identification and can be integrated with the existing computer-vision pipeline.

The proposed landing module further organizes the landing process as a state machine consisting of marker searching, alignment, descent, automatic landing, and landed states. Safety behavior is included for temporary marker loss.

2.6 PX4 EKF2 and Multi-Source State Estimation

2.6.1 Sensor Fusion in Autonomous UAVs

Autonomous UAVs commonly obtain information from multiple sensors. Each sensor has different characteristics and limitations. GPS can provide global position information but may become unavailable, while visual estimation can operate without GPS but may be affected by environmental conditions.

Sensor fusion provides a mechanism for combining these measurements into a consistent estimate of the vehicle state.

2.6.2 PX4 EKF2

PX4 provides an Extended Kalman Filter-based state estimation system through EKF2. The estimator can process information from multiple sources and produce state estimates used by the flight-control system.

For this project, the existing PX4 estimation framework is preferred over implementing a completely independent Kalman Filter. This allows the proposed localization-switching mechanism to focus on determining which localization source should be available or preferred under different conditions while allowing PX4 to perform the underlying state-estimation and fusion tasks.

This design also reduces unnecessary duplication between the research software and the flight controller's existing estimation system.

2.6.3 GPS and VIO Complementarity

GPS and VIO have complementary characteristics. GPS provides an absolute position reference, while VIO can provide relative motion estimation when visual information is available.

The proposed approach therefore investigates an adaptive mechanism that can transition between the two sources according to their estimated reliability. The switching decision considers factors including GPS fix status, position covariance, and the freshness of VIO data.

A hysteresis mechanism is introduced to reduce repeated switching when the system operates close to a predefined threshold. This is particularly important because unstable switching between localization sources could negatively affect the UAV's navigation behavior.

2.7 LoRa for Backup Communication

2.7.1 Communication Requirements

Reliable communication is an important requirement for autonomous UAV systems. The ground system may need to send commands, receive status information, or coordinate the delivery mission.

Internet-based communication can provide relatively flexible and high-level communication between system components. However, its availability depends on network infrastructure. In an emergency environment, this infrastructure may be damaged, overloaded, or unavailable.

Therefore, a backup communication mechanism can increase the robustness of the overall system.

2.7.2 LoRa Communication

LoRa is a long-range wireless communication technology designed for low-power and relatively low-bandwidth communication. These characteristics make it potentially suitable for transmitting small amounts of essential information over relatively long distances.

However, LoRa is not intended to replace a high-bandwidth communication channel. For the proposed UAV system, its role is therefore limited to essential backup communication rather than continuous transmission of large data such as video streams.

2.7.3 Backup Communication Architecture

The proposed software architecture separates the communication transport layer from the higher-level protocol logic. This allows the transport implementation to be replaced without changing the main communication logic.

During the pre-thesis stage, the LoRa communication mechanism is evaluated using a simulated or mock transport rather than a physical LoRa module. The system monitors the state of the primary WebSocket connection and can initiate the backup communication mechanism when the primary channel becomes unavailable.

Physical measurements such as radio range, real air-time latency, and RF interference are outside the scope of the current pre-thesis and are planned for the hardware implementation stage.

2.8 Obstacle Detection

2.8.1 Motivation

In addition to localization and landing, obstacle detection can improve UAV safety by providing information about objects in the flight path.

A practical obstacle detection system should identify potentially dangerous objects and estimate whether they are sufficiently close to require attention. Since the proposed architecture already processes camera data for visual localization, the same camera stream can potentially be reused for obstacle detection.

2.8.2 YOLO-Based Object Detection

YOLO-based object detection provides a computational approach for identifying objects within an image. Instead of processing each object independently through a separate detection process, the YOLO family of models performs object detection using a unified neural-network framework.

For an embedded UAV system, lightweight versions of object-detection models are particularly relevant because companion computers may have limited computational resources.

2.8.3 Monocular Depth Estimation

Object detection alone does not directly provide the distance between the UAV and a detected object. Monocular depth estimation can provide additional information about the relative depth structure of a scene using a single camera.

In this project, Depth Anything V2 is investigated as a method for estimating relative depth. The estimated depth can be combined with object detection results to classify detected objects into approximate categories such as near and far.

However, monocular depth estimation provides relative depth rather than guaranteed metric distance in the current prototype. Its accuracy can also be affected by scene characteristics and lighting conditions.

2.8.4 Current Status

The obstacle detection component remains a preliminary prototype in the current pre-thesis. Initial experiments investigate the classification of objects as near or far using the combination of YOLO and Depth Anything V2.

The module has not yet been integrated as a complete ROS2 node into the main flight-control pipeline. Therefore, it is treated as a supplementary research component rather than one of the primary validated modules of the current system.

2.9 Summary of Related Technologies

The technologies reviewed in this chapter provide complementary capabilities for the proposed autonomous UAV system.

Technology	Main Function	Role in Proposed System
GPS	Global localization	Primary localization source

VIO / SLAM	Relative visual localization	Alternative localization source
ORB-SLAM3	Visual SLAM	GPS-denied localization
Camera Calibration	Camera parameter estimation	Improve visual estimation
ArUco	Fiducial marker detection	Precision landing
PX4 EKF2	State estimation and sensor fusion	Fusion of localization information
LoRa	Long-range low-bandwidth communication	Backup communication
YOLO	Object detection	Obstacle detection prototype
Depth Anything V2	Monocular depth estimation	Relative depth estimation

The reviewed technologies are not intended to operate independently. Instead, they form the technical foundation of an integrated UAV architecture. The main research focus is the interaction between GPS and VIO through an adaptive switching mechanism, while precision landing and backup communication provide additional robustness to the autonomous delivery mission.

The next chapter presents the overall system architecture and explains how these technologies are integrated through ROS2, MAVROS2, PX4, and the ground-side communication infrastructure.

CHAPTER 3: SYSTEM ARCHITECTURE

3.1 Overall System Architecture

The proposed autonomous UAV emergency medical delivery system is designed as a modular software architecture in which the ground-side application, communication infrastructure, companion-computer software, and flight controller are connected through well-defined interfaces.

The overall architecture consists of a Web Application, a Relay Server, a Bridge Node, MAVROS2, and the PX4 flight controller. The vision-based localization, precision landing, and communication functions are implemented as software modules that interact with the flight-control system through the ROS2-based architecture.

The figure should show the following main components:

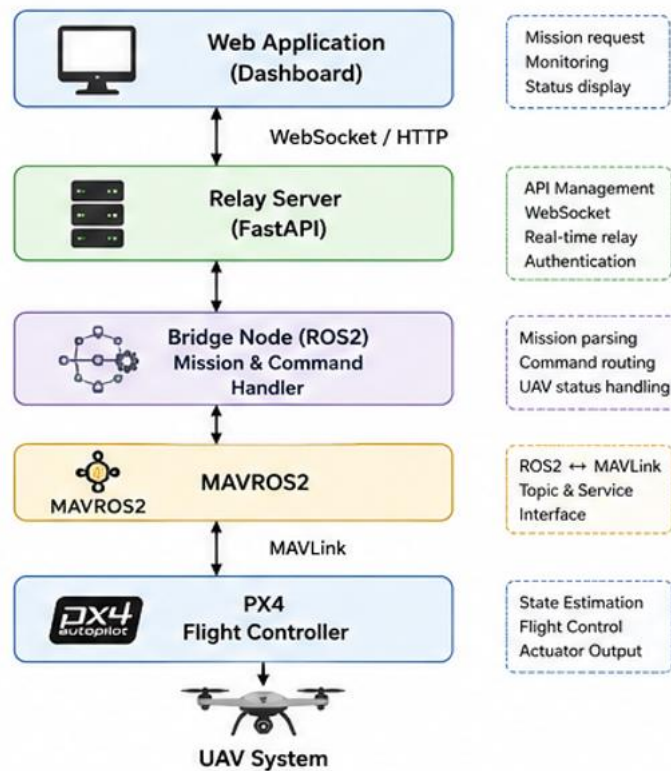


Figure 3.1: Overall System Architecture

The architecture separates high-level mission management from low-level flight control. The Web Application provides the interface for initiating and monitoring delivery missions, while the Relay Server manages communication between external users and the UAV-side software. The Bridge Node converts mission-level information into commands and interfaces with MAVROS2, which provides communication with the PX4 flight-control stack.

This separation allows individual components to be developed and tested independently. It also provides a foundation for replacing simulated components with physical hardware during the full thesis stage.

3.2 Ground-Side Software Architecture

The ground-side portion of the system consists primarily of the Web Application and the Relay Server.

The Web Application provides a user interface through which a medical delivery request can be created and monitored. A request may contain information associated with the delivery mission, including the intended destination and mission status.

The Relay Server acts as an intermediate communication layer between the ground application and the UAV software. It provides APIs for managing delivery requests and WebSocket connections for real-time communication.

The server architecture is designed around several communication endpoints. The patient-side interface can submit requests, while the staff-side interface can manage and dispatch delivery requests. The UAV-side connection is handled separately so that mission commands and UAV status information can be exchanged without directly coupling the Web Application to the flight controller.

This architecture provides an additional abstraction layer between the user interface and the UAV. Consequently, changes to the flight-control implementation do not necessarily require changes to the user-facing application.

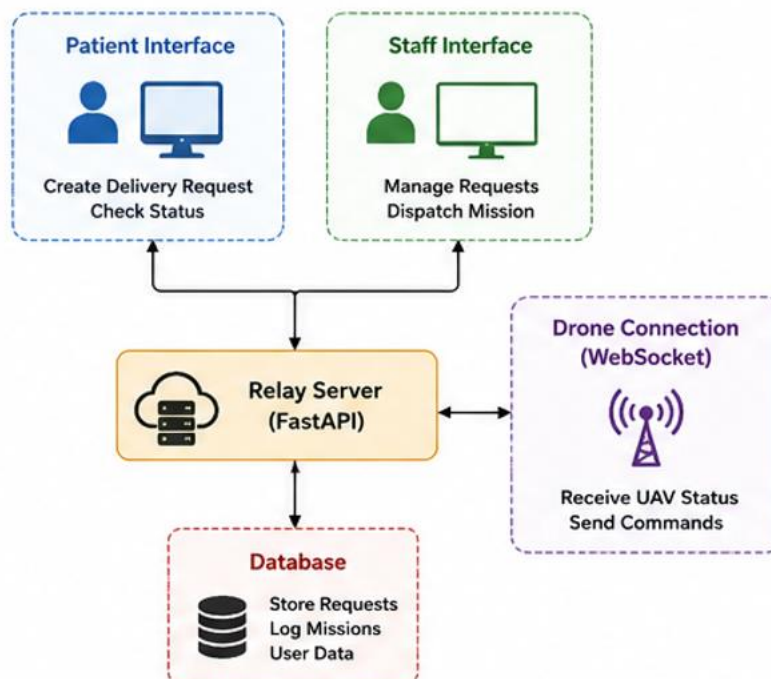


Figure 3.2: Ground-side Communication Architecture

3.3 Companion-Computer Software Architecture

The companion-computer software is responsible for processing information that is not directly handled by the flight controller. In particular, vision processing, localization, landing-target detection, and communication logic can be implemented as independent software modules.

The proposed architecture uses ROS2 as the communication framework between these modules. Each major function is implemented as a ROS2 node or is designed to be integrated as one.

The main functional components include:

- Camera Reader Node;
- VIO/SLAM Node;
- GPS/VIO Switching Logic;
- Precision Landing Node;
- Communication or Bridge Node; and
- Obstacle Detection Prototype.

The modular structure allows each component to have a clearly defined responsibility. For example, the VIO node is responsible for obtaining visual localization information, while the precision landing component processes the visual landing target and generates the corresponding landing behavior.

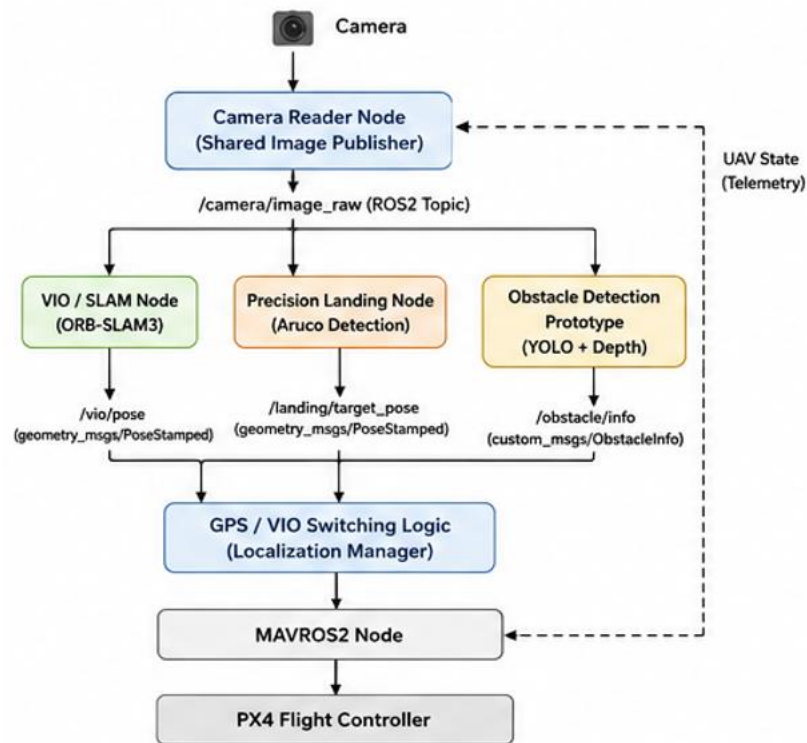


Figure 3.3: ROS2 Node Architecture on Companion Computer

The exact node structure can be refined during implementation. The important design principle is that individual modules communicate through ROS2 interfaces rather than directly depending on the internal implementation of other modules.

3.4 Camera Sharing Architecture

The vision components require access to camera data. Multiple processing modules may need to use the same camera stream, including the VIO/SLAM system, precision landing system, and obstacle detection prototype.

Directly allowing multiple processes to independently access the same physical camera can result in resource conflicts. Depending on the camera driver and operating system, one process may acquire exclusive access to the device while preventing another process from accessing it. To avoid this problem, the proposed architecture introduces a dedicated Camera Reader Node. This node is responsible for acquiring frames from the camera and distributing the resulting image stream to other ROS2 nodes.

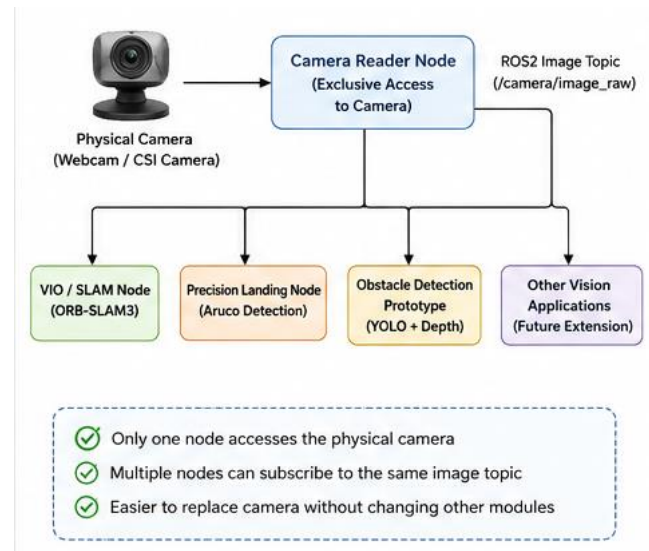


Figure 3.4: Camera Sharing Architecture

This architecture provides two main advantages. First, the physical camera is accessed by only one dedicated component. Second, additional vision-processing modules can subscribe to the same image stream without directly interacting with the camera device.

The design also makes it easier to replace the camera in the future. During the pre-thesis stage, a calibrated webcam can be used as the image source. During the thesis stage, the same software architecture can be adapted to a CSI camera connected to the companion computer.

3.5 ROS2 Communication Architecture

ROS2 is used as the middleware for communication between the software modules. Instead of implementing separate communication mechanisms for each module, ROS2 topics and services provide standardized interfaces.

For example, the VIO module can publish estimated pose information through a ROS2 topic. Other components can subscribe to this information without needing to know the internal implementation of the SLAM system.

One important interface in the proposed architecture is the VIO pose output:

/vio/pose

This interface allows the estimated visual pose to be separated from the internal ORB-SLAM3 implementation.

Similarly, camera images can be distributed through a ROS2 image topic, allowing several processing components to access the same data stream.

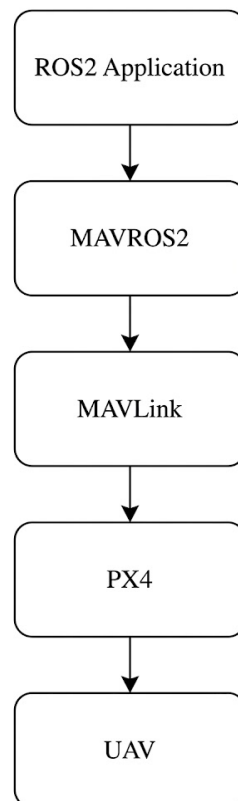
This communication model supports modular development and makes it possible to test individual components independently. It is particularly useful for the current pre-thesis because each functional module can be evaluated in an isolated simulation scenario before complete system integration.

3.6 Communication Between ROS2 and PX4

The flight-control system is based on PX4. Communication between the ROS2 software environment and PX4 is handled through MAVROS2.

MAVROS2 provides an interface between ROS2 applications and the MAVLink-based communication interface used by PX4. Through this interface, higher-level software can obtain vehicle state information and send appropriate commands or setpoints to the flight controller.

The resulting communication chain is:



This separation is important because the ROS2 software does not directly implement the low-level flight-control algorithms. PX4 remains responsible for vehicle stabilization, flight modes, state estimation, and low-level control.

The proposed software modules therefore operate at a higher level and provide information or commands to the flight controller through the established interface.

3.7 Localization Architecture

Localization is one of the most important parts of the proposed architecture because the system must continue operating when GPS becomes unreliable.

Under normal conditions, GPS provides an absolute position reference to the flight-control system. At the same time, the camera-based VIO/SLAM subsystem can provide an alternative estimate of vehicle motion.

The localization architecture therefore contains two main sources:

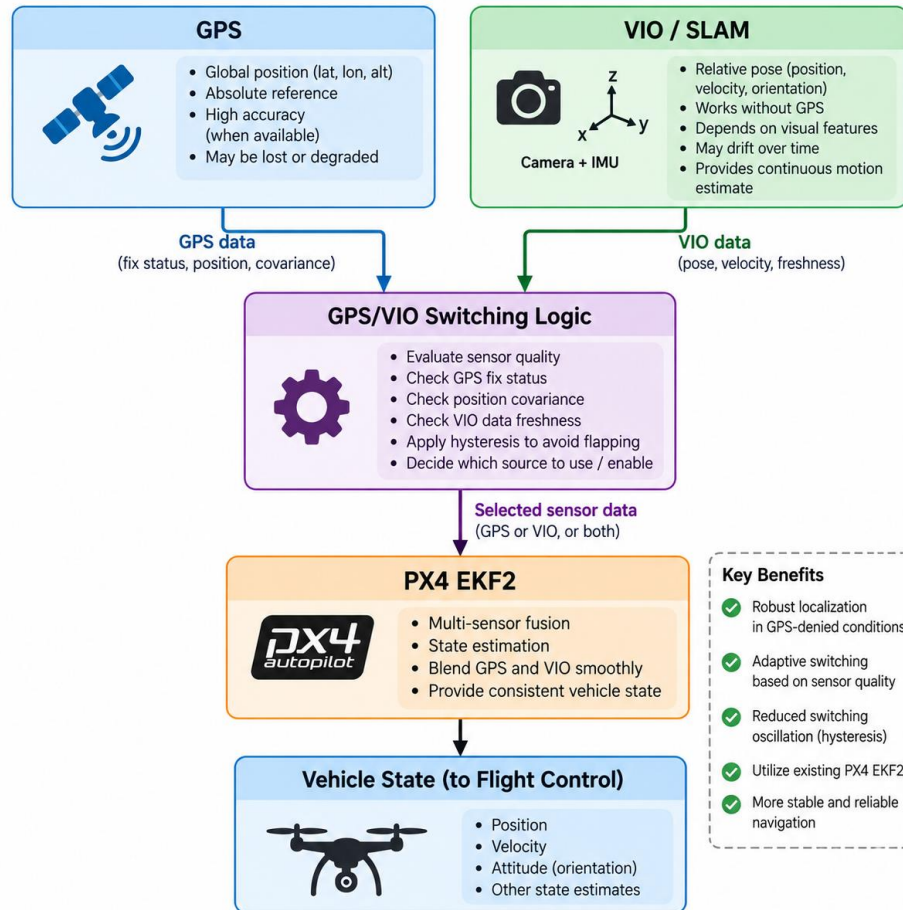


Figure 3.5: Localization Architecture

The switching mechanism evaluates the availability and quality of the localization sources. The detailed switching algorithm is described in Chapter 5.

The architecture intentionally separates the decision about localization-source availability from the low-level state-estimation process. PX4 EKF2 remains responsible for state estimation and sensor fusion, while the proposed software determines when GPS or VIO information should be considered appropriate.

3.8 Precision Landing Architecture

Precision landing is treated as a separate stage of the UAV mission. During the main navigation phase, the UAV can use its normal localization system to approach the target area. When the

landing phase begins, the camera-based landing system searches for the designated ArUco marker.

Once the marker is detected, its relative position with respect to the camera is estimated. The resulting alignment information is used to guide the UAV toward the center of the landing target.

The landing process follows a state-based architecture:

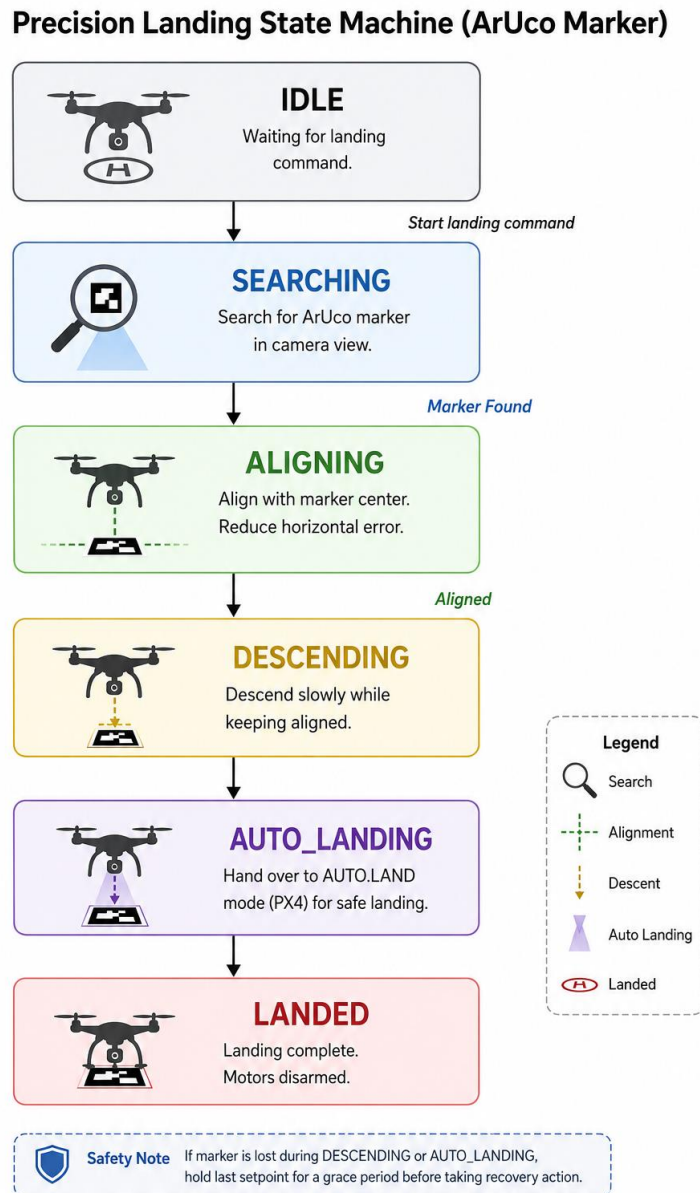


Figure 3.6: Precision State Machine

A temporary loss of the marker does not immediately terminate the landing process. Instead, the system can maintain the latest valid setpoint for a limited grace period. This behavior is intended to reduce unnecessary oscillations caused by short-term visual detection failures.

The complete precision-landing implementation and experimental results are presented in Chapter 6.

The system uses a primary network communication channel for normal operation. However, because the target application involves emergency environments, the architecture also considers a backup communication mechanism.

The primary communication path uses the network-based communication infrastructure. LoRa is considered as the secondary channel for essential low-bandwidth information.

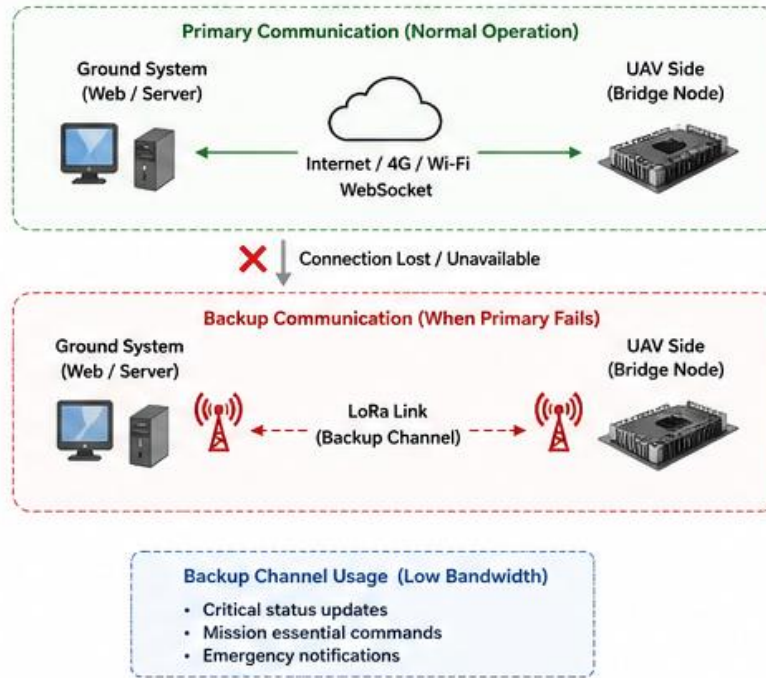


Figure 3.7: Primary and Backup Communication Architecture

The communication architecture separates the transport mechanism from the application-level protocol. This allows a simulated transport to be used during the pre-thesis while keeping the higher-level communication logic independent of the eventual physical LoRa hardware.

The detailed communication fallback mechanism and its testing procedure are presented in Chapter 7.

3.9 Development and Simulation Environment

The proposed system is developed and evaluated using a combination of simulation and real vision hardware.

PX4 Software-In-The-Loop (SITL) is used to simulate the flight controller and UAV behavior. A simulator such as Gazebo or jMAVSim can provide the simulated vehicle environment, while ROS2 and MAVROS2 provide the communication framework between the simulated vehicle and the software modules.

A real webcam is used for vision-related experiments. The camera is calibrated before being used by the visual-processing pipeline. This allows the project to evaluate the behavior of the vision algorithms using real camera data while keeping the UAV itself within a simulation environment.

The overall pre-thesis environment can therefore be summarized as follows:

Component	Technology	Purpose
Flight Controller	PX4	UAV flight control and state estimation
Vehicle Simulation	PX4 SITL + Simulator	Simulated UAV operation
Middleware	ROS2	Inter-node communication
MAVLink Interface	MAVROS2	ROS2–PX4 communication
Visual Localization	ORB-SLAM3	GPS-denied localization
Camera	Real Webcam	Visual input
Camera Calibration	Real calibration data	Camera parameter estimation
Landing Detection	ArUco	Precision landing
Primary Communication	WebSocket / Network	Mission communication
Backup Communication	LoRa architecture	Communication fallback
Obstacle Detection	YOLO + Depth Anything V2	Prototype obstacle perception

The use of simulation provides a controlled environment in which individual system functions can be tested without the safety risks associated with real UAV flight. It also makes it possible to reproduce specific conditions, such as GPS failure or network disconnection, and evaluate the system response.

However, simulation cannot fully reproduce the physical characteristics of real flight. Sensor noise, vibration, real GPS interference, camera motion, radio propagation, and environmental disturbances may behave differently on physical hardware. These limitations are considered in Chapter 10.

3.10 System Design Considerations

The architecture is designed around modularity, testability, and future hardware deployment.

First, modularity allows each major function to be developed independently. The localization, landing, and communication modules can be tested without requiring the complete system to operate simultaneously.

Second, the architecture supports isolated testing. This is particularly important during the pre-thesis stage because the current implementation is still under development. Testing individual modules allows specific variables and failure conditions to be analyzed more clearly.

Third, the architecture is designed with future hardware deployment in mind. The current webcam-based vision pipeline can later be connected to a physical CSI camera, while the simulated PX4 environment can eventually be replaced by a real Pixhawk-based flight controller. Similarly, the mock LoRa transport can later be replaced by a physical LoRa transceiver without fundamentally changing the higher-level protocol design.

Finally, the architecture attempts to minimize unnecessary duplication of flight-control functionality. PX4 remains responsible for low-level flight control and state estimation, while ROS2-based modules provide higher-level perception, decision-making, and mission-level functionality.

CHAPTER 4: GPS-DENIED LOCALIZATION USING VIO/SLAM

4.1 ORB-SLAM3 Fundamentals

Global Positioning System (GPS) provides an absolute position reference for UAV navigation, but its availability cannot always be guaranteed. When GPS information becomes unavailable or unreliable, the UAV requires an alternative localization source to estimate its motion. In this project, Visual-Inertial Odometry (VIO) and Visual SLAM are investigated as the primary alternatives for GPS-denied localization.

Visual SLAM estimates the motion of a camera while simultaneously constructing a representation of the observed environment. Instead of relying on satellite-based positioning, the system uses visual information obtained from the camera to estimate changes in the camera pose over time.

ORB-SLAM3 is selected as the Visual SLAM framework for this project. The system provides the main components required for visual localization, including tracking, mapping, and loop closing. The use of an existing SLAM framework allows the project to focus on integration with the UAV software architecture rather than implementing a complete SLAM algorithm from the beginning.

The main processing stages relevant to this project are visual feature extraction and tracking, camera pose estimation, map construction, and loop-closure processing.

4.1.1 Tracking

The tracking process receives image frames from the camera and extracts visual features that can be matched between consecutive frames. By establishing correspondences between observations, the system estimates the relative motion of the camera.

The estimated camera pose provides information about the motion of the UAV in the visual reference frame. Continuous tracking is therefore essential for maintaining localization when GPS information is unavailable.

Tracking performance depends on several factors, including image quality, illumination, scene texture, camera motion, and camera calibration. Environments containing sufficient visual features generally provide more reliable tracking than environments with large textureless regions.

4.1.2 Mapping

While tracking estimates the camera motion, the mapping component maintains a representation of the observed environment. Visual features that are observed from multiple viewpoints can be incorporated into the map as spatial landmarks.

The map allows subsequent camera observations to be related to previously observed locations. This provides a basis for maintaining localization over a longer trajectory rather than estimating motion independently for every pair of consecutive frames.

4.1.3 Loop Closing

Visual odometry can accumulate estimation errors over time. Loop closing provides a mechanism for recognizing previously visited areas and correcting accumulated trajectory errors.

When the system identifies that the camera has returned to a previously mapped region, the corresponding information can be used to improve the consistency of the estimated trajectory and map.

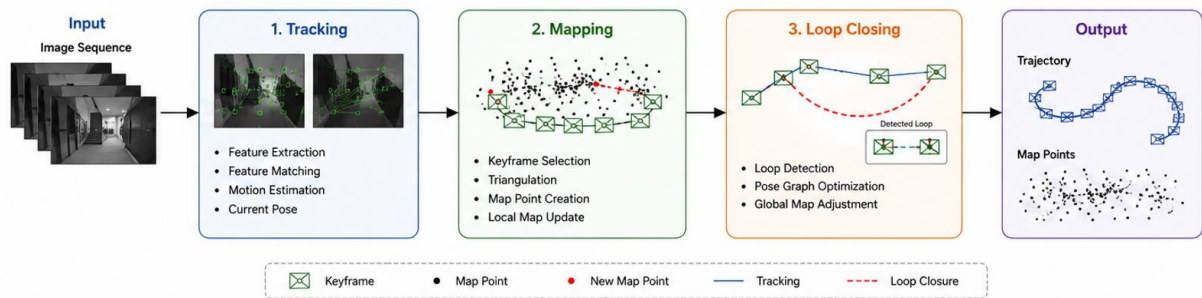


Figure 4.1: Simplified ORB-SLAM3 Processing Pipeline

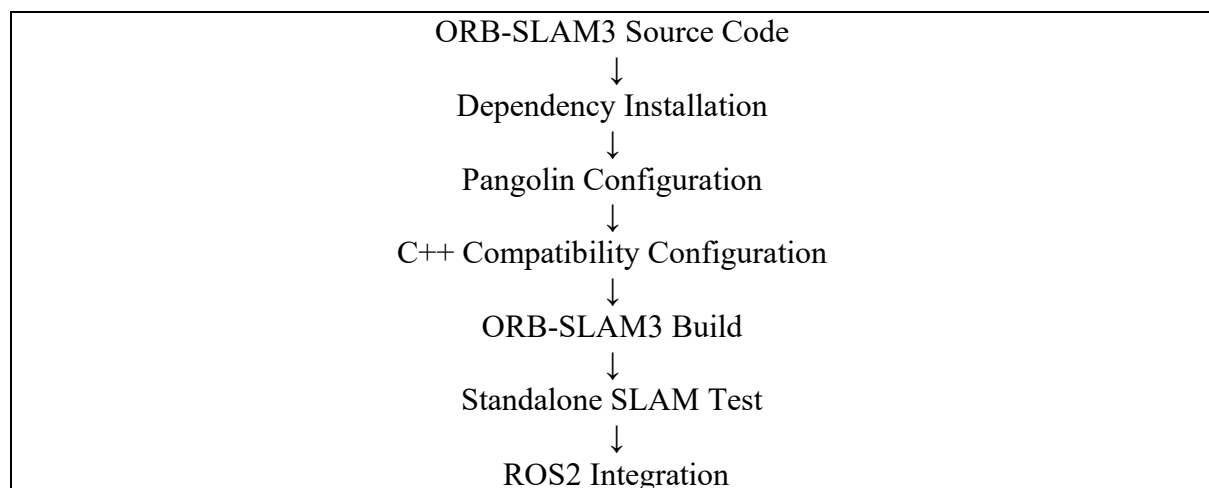
4.2 ORB-SLAM3 Deployment and Implementation

The ORB-SLAM3 component is integrated into the development environment as the visual localization engine of the proposed system. The implementation requires building the required software dependencies and configuring the system to operate with the selected C++ environment.

One of the important components used during development is Pangolin. Pangolin provides the visualization interface used by ORB-SLAM3 to display information such as the estimated camera trajectory, keyframes, and reconstructed map.

The ORB-SLAM3 source code and its dependencies are built from source to provide the required compatibility with the project's software environment. Particular attention is given to C++ compatibility because the selected development environment requires the appropriate language standard for successful compilation and integration.

The deployment process can be summarized as:



Before integrating ORB-SLAM3 with ROS2, the SLAM system is first tested independently. This allows compilation, visualization, camera input, and basic tracking behavior to be verified separately from the ROS2 communication layer.

This staged implementation reduces the difficulty of debugging because errors related to SLAM processing can be distinguished from errors related to ROS2 integration.

4.3 Camera Calibration

Camera calibration is an important step before using a real camera for visual localization. The geometric relationship between a three-dimensional point and its corresponding image position depends on the intrinsic parameters of the camera and its lens distortion characteristics.

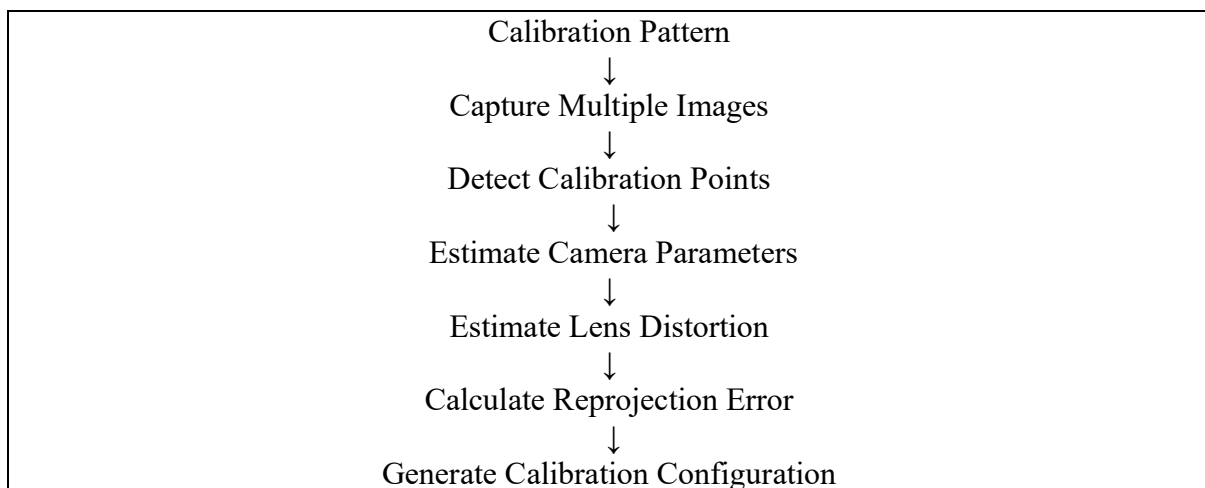
The calibration process estimates parameters such as focal length, principal point, and distortion coefficients. These parameters are subsequently used by the visual localization system to interpret image measurements correctly.

In this project, a real webcam is used as the visual input. The camera is calibrated before being integrated into the VIO/SLAM pipeline. This allows the experiment to use actual camera characteristics rather than relying only on idealized or default parameters.

4.3.1 Calibration Procedure

The calibration procedure uses multiple observations of a calibration pattern from different orientations and positions. The detected calibration points are used to estimate the camera's intrinsic parameters and distortion coefficients.

The general process is:



The resulting parameters are then provided to ORB-SLAM3 so that image observations can be processed using the calibrated camera model.

4.3.2 Reprojection Error

Reprojection error is used as an indicator of how well the estimated camera model represents the observed calibration points. The estimated three-dimensional calibration points are projected back onto the image plane, and the difference between the projected points and the observed points is measured.

A lower reprojection error generally indicates a better correspondence between the estimated camera model and the calibration observations. However, reprojection error alone does not guarantee good SLAM performance because tracking is also affected by image quality, scene texture, motion, and lighting conditions.

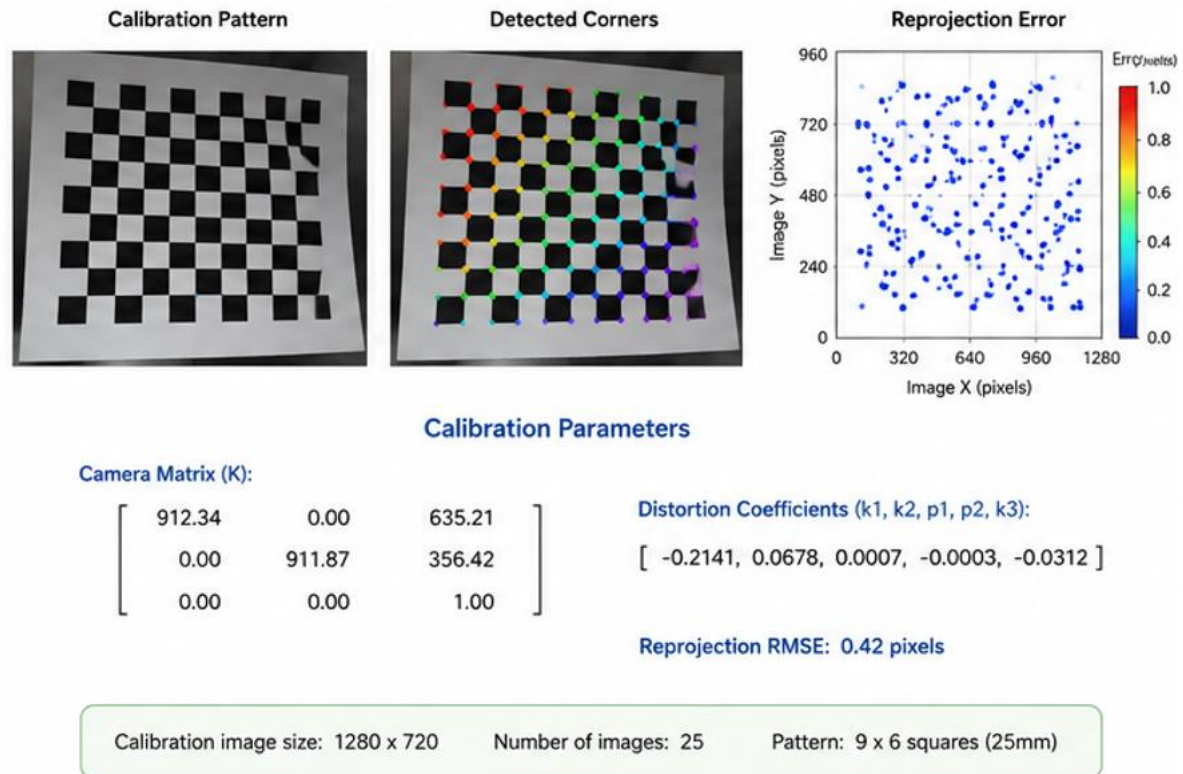


Figure 4.2: Camera Calibration Result

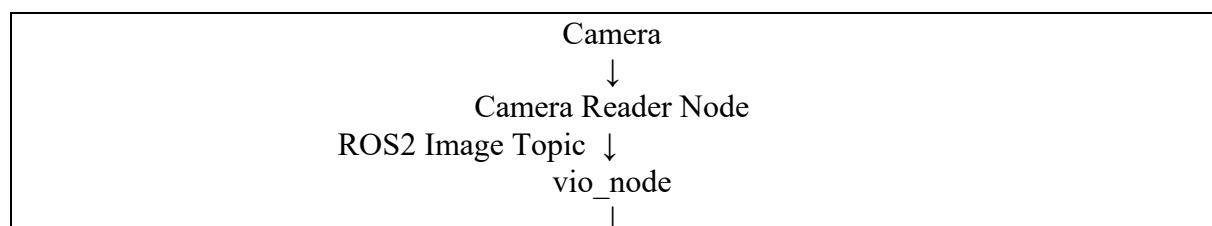
4.4 ROS2 Integration

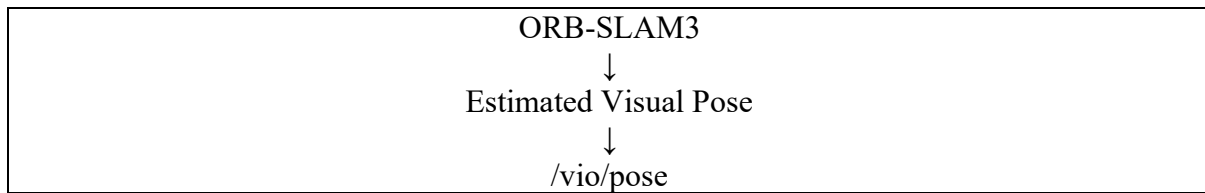
After verifying the standalone ORB-SLAM3 implementation, the visual localization system is integrated into the ROS2 architecture.

The main purpose of the ROS2 integration is to make the estimated visual pose available to other components of the UAV system. Instead of allowing the rest of the system to directly interact with the internal ORB-SLAM3 implementation, a dedicated ROS2 node is used to process the visual localization pipeline.

The main component is referred to as the `vio_node`. Its responsibility is to receive camera data, provide the required input to the visual localization system, and publish the resulting pose information through ROS2.

The resulting architecture can be represented as:





The `/vio/pose` interface provides a separation between the SLAM implementation and the rest of the UAV system. Other ROS2 components can subscribe to the pose information without needing to directly access the internal data structures of ORB-SLAM3.

This design is particularly useful for the subsequent GPS/VIO switching module. The switching logic can monitor the availability and freshness of the `/vio/pose` data while remaining independent of the internal implementation of the SLAM algorithm.

4.4.1 Shared Camera Input

The camera input is designed to support multiple vision-processing modules. Instead of allowing every processing module to directly access the physical camera, a Camera Reader Node is used as the common source of image data.

This architecture prevents multiple processes from competing for direct access to the same camera device. It also allows additional vision modules to subscribe to the same camera stream through ROS2.

For example, the same image stream can potentially be used by:

- ORB-SLAM3 for visual localization;
- ArUco detection for precision landing; and
- The obstacle detection prototype.

This approach also simplifies future hardware replacement because the camera source can be changed without fundamentally modifying the higher-level processing nodes.

4.5 Localization Results

The performance of the visual localization system is evaluated by observing the stability of visual tracking and the resulting reconstructed map.

The evaluation focuses on qualitative and structural indicators provided by ORB-SLAM3, including the estimated camera trajectory, keyframes, and map points.

A successful tracking sequence should produce a continuously estimated camera trajectory without frequent tracking failures. The reconstructed map should also contain a sufficient number of spatial landmarks to support continued localization.

The following aspects are considered during evaluation:

1. **Tracking stability** – whether the system can maintain visual tracking over the tested sequence.
2. **Camera trajectory** – whether the estimated trajectory follows the expected camera motion.
3. **Keyframes** – whether the system generates keyframes throughout the sequence.
4. **Map points** – whether sufficient visual landmarks are reconstructed.

5. **Visualization consistency** – whether the Pangolin viewer provides a coherent representation of the estimated trajectory and map.

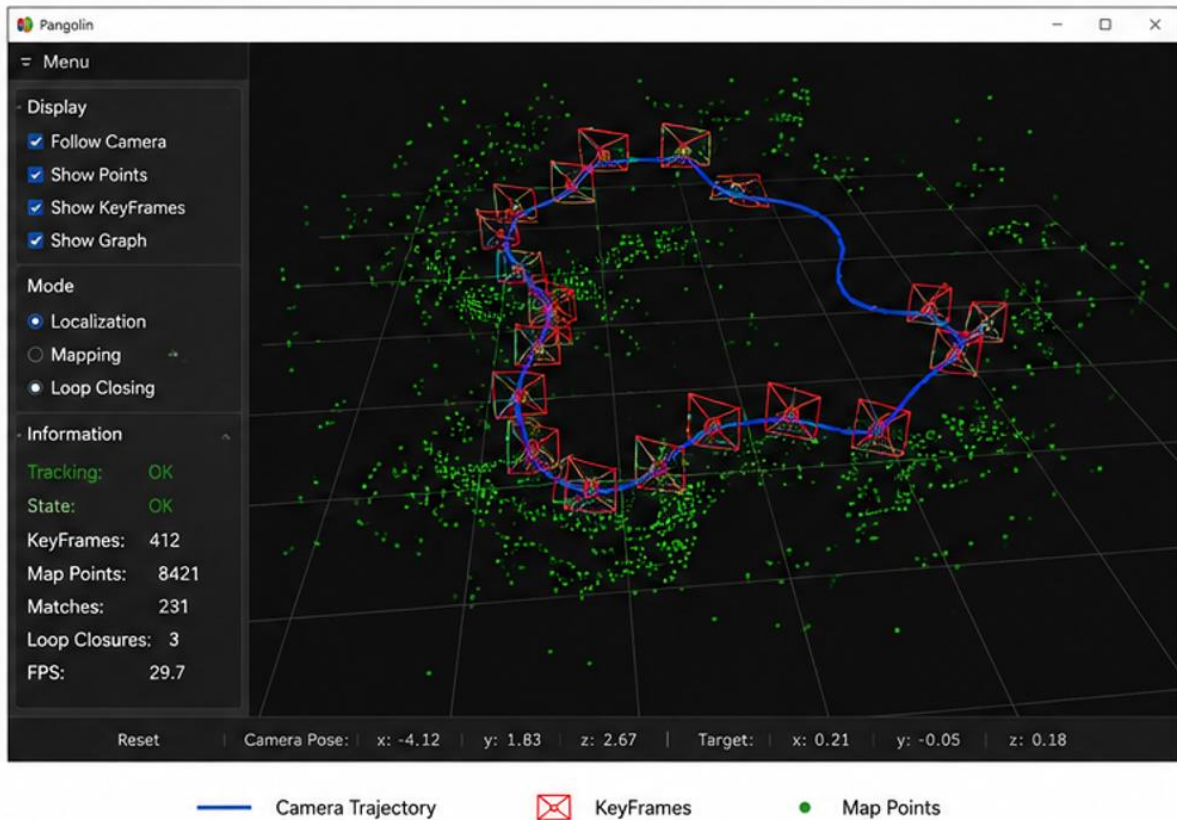


Figure 4.3: ORB-SLAM3 Result

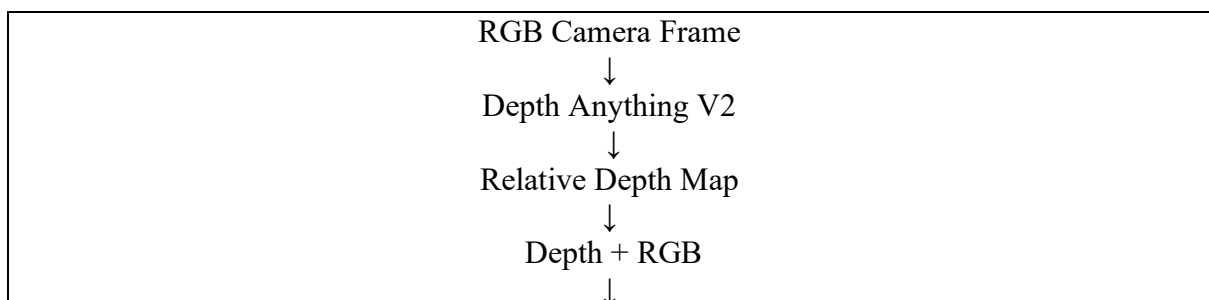
4.6 Pseudo-Dense Reconstruction Using Depth Anything V2

In addition to sparse visual mapping from ORB-SLAM3, the project investigates a pseudo-dense reconstruction approach using monocular depth estimation.

ORB-SLAM3 provides a sparse representation consisting primarily of tracked visual features and reconstructed map points. While this representation is sufficient for localization, a denser representation of the scene may provide additional information for perception-related tasks.

Depth Anything V2 is investigated as a monocular depth-estimation model that can generate a depth representation from a single RGB image. The resulting depth information can be combined with image data to generate a pseudo-dense representation of the observed environment.

The processing pipeline can be summarized as:



The purpose of this extension is to investigate whether monocular depth information can complement the sparse representation obtained from ORB-SLAM3.

However, the output of the current monocular depth-estimation approach should be interpreted carefully. The estimated depth is relative rather than a directly calibrated metric distance in meters. Therefore, the resulting reconstruction should not be treated as a precise three-dimensional metric map.

This limitation is particularly important for UAV applications because obstacle avoidance and distance-based control require reliable metric information. Further calibration and validation would therefore be required before using the depth estimation output directly for flight control.

At the current pre-thesis stage, this component is considered an extension of the visual perception pipeline rather than a replacement for ORB-SLAM3 localization.

4.7 Discussion

The results of this module demonstrate the feasibility of using camera-based visual localization as an alternative information source for GPS-denied UAV operation. ORB-SLAM3 provides the required visual tracking and mapping functionality, while ROS2 integration makes the estimated pose available to the remaining components of the proposed UAV architecture.

Camera calibration is an essential preprocessing step because the visual localization system depends on the camera model. The use of a real calibrated webcam also provides a more realistic vision input than an idealized camera model, although it does not reproduce all of the sensor characteristics of a camera mounted on a flying UAV.

The current implementation also demonstrates the importance of separating visual processing from camera acquisition. By using a dedicated Camera Reader Node, multiple vision modules can access the same image stream without independently controlling the physical camera.

The visual localization module provides the foundation for the next stage of the project. In particular, the estimated VIO pose can be monitored together with GPS information to determine which localization source should be used under different operating conditions.

The following chapter therefore focuses on the adaptive mechanism for switching between GPS and VIO. Rather than simply replacing GPS whenever it becomes unavailable, the proposed system evaluates localization quality and uses quantitative conditions to determine when a transition should occur.

CHAPTER 5: ADAPTIVE GPS-to-VIO LOCALIZATION SWITCHING

5.1 Motivation for Conditional Switching

GPS is the primary source of global position information for the UAV during normal operation. However, GPS may become unreliable or unavailable in certain environments. As described in Chapter 4, VIO can provide an alternative source of localization without depending on GPS.

A simple approach would be to immediately switch from GPS to VIO whenever GPS is lost. However, this approach may cause unstable behavior when the GPS signal temporarily becomes weak and then recovers. Repeated switching between GPS and VIO can introduce discontinuities and reduce the reliability of the navigation system.

Therefore, this project proposes a condition-based switching mechanism that evaluates the quality of both localization sources before changing the active source.

The objective is not to replace GPS permanently with VIO, but to allow the system to select an appropriate localization source according to the current operating conditions.

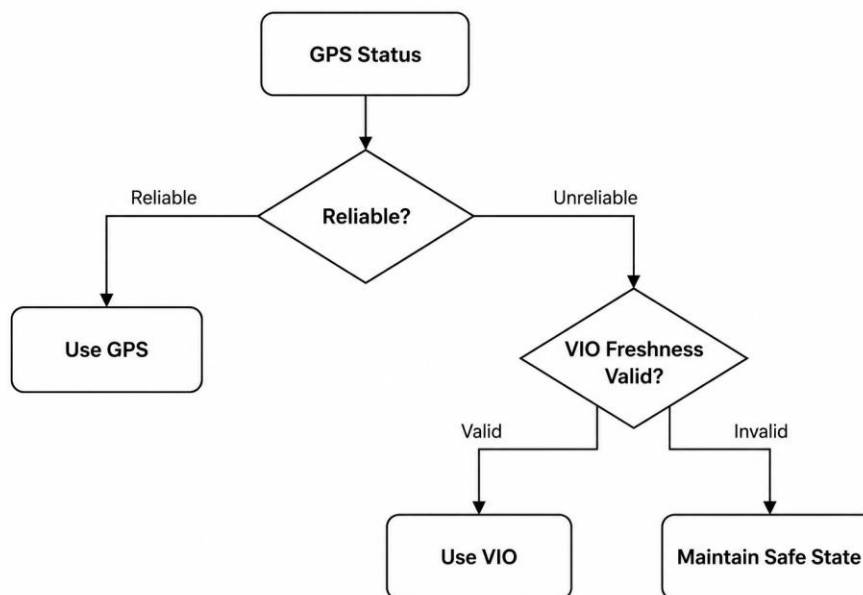
5.2 Quantitative Switching Conditions

The switching mechanism evaluates three main indicators:

1. **GPS fix status** – determines whether a valid GPS solution is currently available.
2. **GPS position covariance** – provides an indication of the estimated uncertainty of the GPS position.
3. **VIO data freshness** – determines whether the latest VIO pose is recent enough to be considered valid.

These conditions are used to determine whether the system should remain with GPS, switch to VIO, or return from VIO to GPS.

The general decision process is:



When GPS becomes unreliable and valid VIO data is available, the system can transition toward VIO-based localization. When GPS becomes reliable again, the system can subsequently return to GPS.

The exact threshold values used for covariance and VIO freshness are implementation parameters and should be reported according to the actual values used during testing.

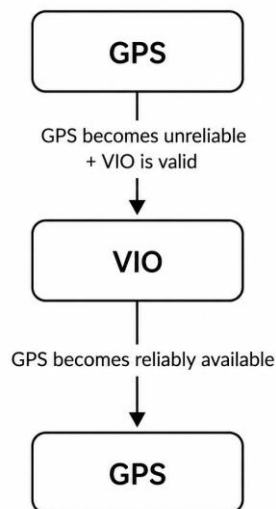
5.3 Hysteresis for Stable Switching

A major problem with threshold-based switching is flapping, where the system repeatedly changes between GPS and VIO because the measured signal quality fluctuates around the switching threshold.

For example, if GPS quality is close to the defined threshold, a small variation in the covariance may cause the system to switch to VIO. If the covariance immediately improves again, the system may switch back to GPS. Repeating this process can make the navigation source unstable.

To prevent this behavior, the proposed system introduces hysteresis. Different conditions are used for entering and leaving the VIO state rather than using a single threshold for both directions.

Conceptually:



The system therefore requires stronger evidence before changing the active localization source. This reduces unnecessary transitions caused by short-term fluctuations.

The hysteresis mechanism is particularly important for UAV operation because localization should remain stable even when sensor quality changes near the decision boundary.

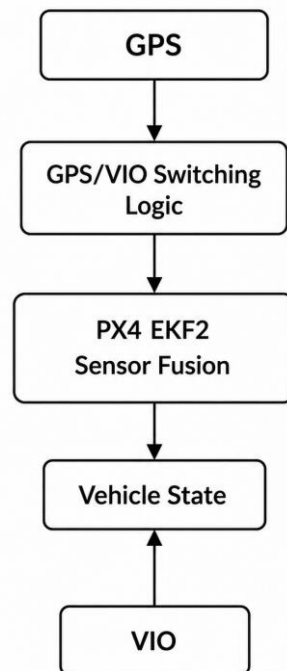
5.4 Integration with PX4 EKF2

The switching mechanism is integrated with the existing state-estimation architecture of PX4. Instead of implementing a separate Kalman Filter, the project makes use of the EKF2 estimator already provided by PX4.

PX4 EKF2 is responsible for combining available sensor measurements and producing the vehicle state estimate required by the flight-control system. The proposed switching logic therefore focuses on determining when GPS or VIO information should be considered valid and appropriate.

This design has two advantages. First, it avoids duplicating the state-estimation functionality already implemented in PX4. Second, it allows the proposed contribution to focus specifically on the decision-making problem of selecting between localization sources.

The overall concept can be represented as:



The switching mechanism and EKF2 therefore have different responsibilities. The switching logic evaluates localization-source conditions, while EKF2 performs the underlying state estimation and sensor fusion.

This approach also makes the system easier to extend because additional localization sources can potentially be incorporated without replacing the complete flight-controller estimator.

5.5 SITL Testing and Results

The GPS/VIO switching mechanism was tested using the PX4 Software-In-The-Loop (SITL) environment to verify the behavior of the localization-source selection logic. The first part of the test focused on the condition where GPS was unavailable while VIO remained valid. As shown in the first test output, the switching node reported `gps_ok: False` and `vio_ok: True`. Based on these conditions, the node changed the active localization source from `NONE` to `VIO`. The following status messages continued to show `gps_ok: False`, `vio_ok: True`, and `Active Source: VIO`, indicating that VIO remained selected while GPS was unavailable.

```
[WARN] [1788259693.822926349] [gps_vio_switching_node]: SWITCHING LOCALIZATION SOURCE: NONE -> VIO
[INFO] [1788259694.321631982] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788259694.821567782] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788259695.321691649] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788259695.821473399] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788259696.321501390] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788259696.821353340] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
```

After the system remained in the VIO state, GPS became available again during the test. The switching node then reported `gps_ok: True` while `vio_ok` remained `True`. Although both sources were valid at this point, VIO was still the active source initially. The switching logic subsequently detected the restored GPS condition and performed the transition from VIO to GPS. The following status messages confirmed that GPS became the active localization source and remained selected afterward.

```
[INFO] [1788260223.505341150] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788260224.005832700] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788260224.507736916] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788260225.007313266] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: VIO
[WARN] [1788260225.011759483] [gps_vio_switching_node]: SWITCHING LOCALIZATION SOURCE: VIO -> GPS
[INFO] [1788260225.506232250] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
[INFO] [1788260226.005597783] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
[INFO] [1788260226.506399925] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
[INFO] [1788260227.005103650] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
```

Overall, the SITL test demonstrates the intended two-way localization-source switching behavior. When GPS was unavailable and valid VIO data was present, the system selected and maintained VIO as the active source. When GPS became available again, the system successfully switched back to GPS. This confirms that the implemented switching node can respond to changes in localization-source availability and manage the transition between GPS and VIO as intended.

5.6 Limitations

The main limitation of the current evaluation is that GPS failure is reproduced using a simulated environment rather than real GPS degradation or interference. Therefore, the behavior observed in SITL may not completely represent the behavior of a physical UAV operating under real signal interference, multipath effects, or sensor noise.

In addition, the current evaluation focuses primarily on verifying the switching logic and its integration with the PX4 software stack. A complete evaluation of localization accuracy during real GPS-to-VIO transitions requires flight testing with physical sensors and hardware.

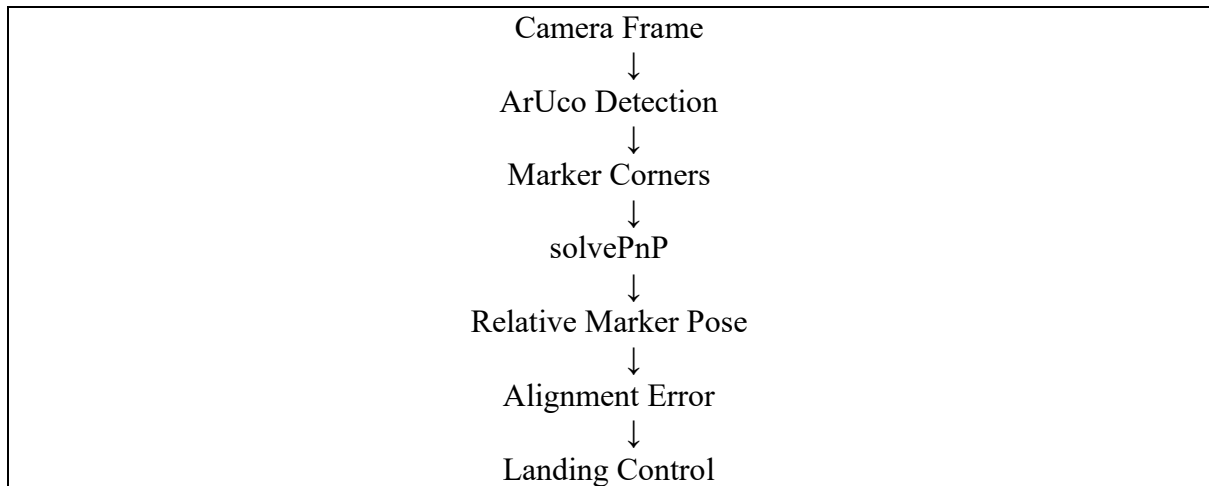
These limitations will be addressed during the full thesis stage through real-hardware deployment and flight experiments.

CHAPTER 6: PRECISION LANDING USING ARUCO MARKER

6.1 Precision Landing Method

The precision landing module is designed to improve the final landing accuracy of the UAV by using a visual reference instead of relying only on the navigation position estimate. An ArUco marker is placed at the designated landing position and detected using the onboard camera. When the marker is detected, its corner coordinates are extracted from the image and used to estimate the relative pose between the camera and the marker.

The marker pose is estimated using the solvePnP method. The resulting translation information provides the relative position of the marker with respect to the camera, which can be used to determine the horizontal alignment error between the UAV and the landing target. The general processing flow is:



The purpose of this process is to ensure that the UAV is sufficiently aligned with the landing marker before proceeding to the final descent.

6.2 Landing Control State Machine

The landing procedure is organized as a state machine to control the UAV through different stages of the visual landing process. The main states are IDLE, SEARCHING, ALIGNING, DESCENDING, AUTO_LANDING, and LANDED.

In the IDLE state, the precision-landing module is inactive. Once the landing procedure is activated, the system enters SEARCHING and attempts to detect the ArUco marker. After successful detection, the system enters ALIGNING and adjusts the UAV position toward the marker center. When the alignment condition is satisfied, the UAV proceeds to DESCENDING. The final stage is handled through AUTO_LANDING, after which the system enters the LANDED state.

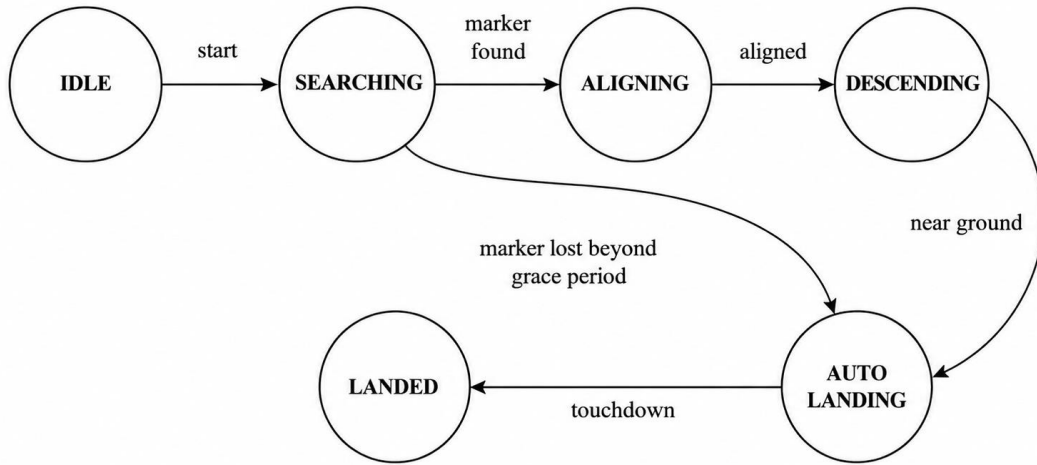


Figure 6.1: State Machine of the ArUco-based Precision Landing Process

6.3 Safety Mechanism

Visual marker detection can temporarily fail because of motion blur, lighting changes, partial occlusion, or other image-processing limitations. Immediately stopping or changing the landing behavior when the marker disappears for a short period could therefore cause unnecessary instability.

To address this problem, the system uses a short grace period when the marker is temporarily lost. During this period, the system can maintain the last valid landing setpoint while continuing to search for the marker. If the marker is detected again within the allowed period, the landing process continues normally.

If the marker remains unavailable beyond the allowed period, the system should avoid continuing the descent without reliable visual information and instead transition to a predefined safe behavior. The final touchdown is handed over to the PX4 AUTO.LAND procedure, reducing the need for the vision-based controller to directly handle the complete touchdown phase.

```

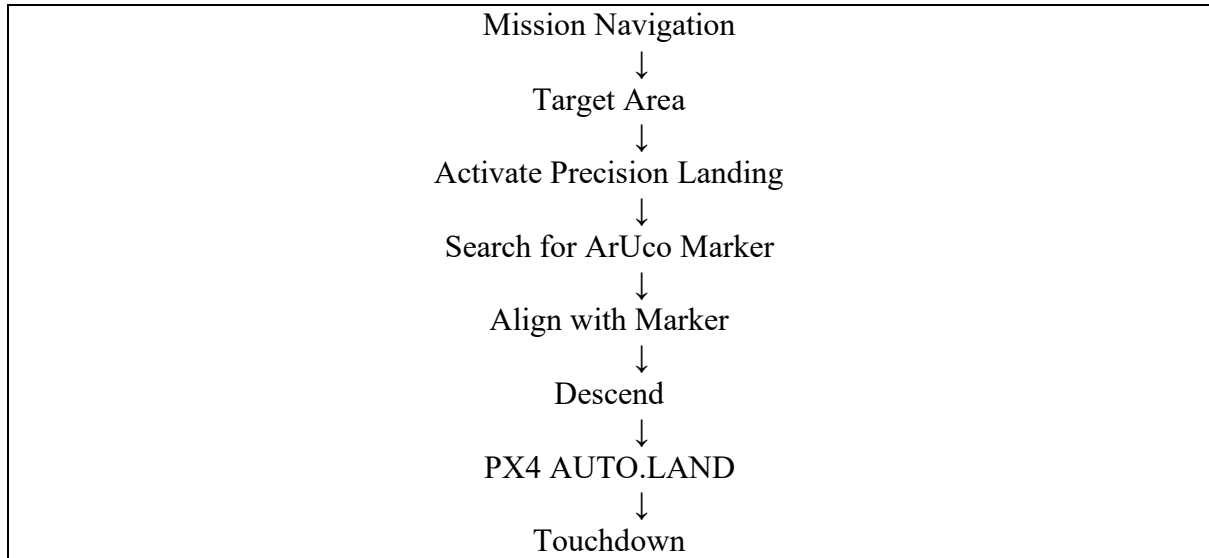
[INFO] [1788409694.525629711] [landing_controller_node]: Activated by bridge node. Starting search for marker.
[INFO] [1788409697.743461865] [landing_controller_node]: Marker acquired. Switching to ALIGNING.
[INFO] [1788409697.771953563] [landing_controller_node]: Aligned over marker. Switching to DESCENDING.
[ERROR] [1788409705.672985329] [landing_controller_node]: Marker lost beyond grace period. Stopping (zero velocity).
[ERROR] [1788409705.770963629] [landing_controller_node]: Marker lost beyond grace period. Stopping (zero velocity).
[ERROR] [1788409705.872013929] [landing_controller_node]: Marker lost beyond grace period. Stopping (zero velocity).
[ERROR] [1788409705.976407628] [landing_controller_node]: Marker lost beyond grace period. Stopping (zero velocity).
[ERROR] [1788409706.068741528] [landing_controller_node]: Marker lost beyond grace period. Stopping (zero velocity).
[INFO] [1788409719.368854343] [landing_controller_node]: Close enough to marker (Z < threshold). Handing off to FCU AUTO.LAND - avoid
ing OFFBOARD near ground to prevent ground-effect turbulence.
[INFO] [1788409719.495492977] [landing_controller_node]: AUTO.LAND mode accepted by PX4.
  
```

Figure 6.2: Precision Landing State Transitions and Safety Handling

6.4 Handoff from Navigation to Precision Landing

Precision landing is used only during the final stage of the mission. During normal flight, the UAV follows its planned trajectory using the navigation system described in the previous chapters. When the UAV reaches the target area, the precision-landing procedure is activated.

The handoff can be summarized as:



This approach separates the responsibilities of navigation and landing. The navigation system brings the UAV to the target area, while the visual landing module provides the final alignment required for accurate landing.

6.5 Experimental Results

The precision-landing module was evaluated using camera calibration, marker-distance measurements, and an end-to-end test of the landing workflow. Camera calibration was performed to obtain the intrinsic parameters required for accurate pose estimation. The detected ArUco marker was then used to estimate its relative position and distance from the camera.

The experimental evaluation focused on marker detection, pose estimation, and the ability of the system to provide information required for the final landing process. The measured distance can be compared with the actual reference distance to evaluate the accuracy of the pose estimation.

The distance error is calculated as:

$$E_d = |d_{\text{measured}} - d_{\text{reference}}|$$

where d_{measured} is the distance estimated from the camera and $d_{\text{reference}}$ is the actual measured distance.

For the end-to-end test, the complete workflow was evaluated from mission dispatch and UAV navigation to marker detection, alignment, descent, and touchdown. The results obtained from these experiments provide an initial validation of the precision-landing module.

```

[INFO] [1788409718.170480218] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.270970977] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.370546318] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.476418210] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.569334735] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.671237727] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.772419652] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.874453835] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409718.970695327] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409719.071232885] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409719.171710752] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409719.271263777] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409719.379059343] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788409719.475187627] [uav_cloud_bridge]: Precision landing status: AUTO_LANDING
[INFO] [1788409719.500706085] [uav_cloud_bridge]: Precision landing status: LANDED
[INFO] [1788409719.571875452] [uav_cloud_bridge]: Precision landing status: LANDED
[INFO] [1788409719.734456702] [uav_cloud_bridge]: Precision landing succeeded.
[INFO] [1788409719.742363627] [uav_cloud_bridge]: Mission timeline finished.

```

Figure 6.3: Precision Landing Completion and Successful Touchdown

6.6 Limitations

The current evaluation has a limitation caused by the testing configuration. A real webcam is used for visual processing, while the UAV itself is represented by a simulated vehicle. Therefore, the physical ArUco marker does not automatically move according to the simulated UAV position.

This setup is sufficient for validating the marker detection, camera calibration, pose-estimation, and landing-control logic, but it does not fully reproduce the behavior of a physical UAV approaching a real landing marker.

Further testing with a physical UAV, onboard camera, and real landing environment is therefore required to evaluate the complete precision-landing performance under real flight conditions.

CHAPTER 7: BACKUP COMMUNICATION USING LoRa

7.1 Role of LoRa as a Backup Communication Channel

The UAV system primarily uses a WebSocket-based communication channel for exchanging information between the ground system and the UAV. However, the main communication link may become unavailable or unstable because of Internet connectivity problems or network interruptions.

To improve communication reliability, LoRa is introduced as a backup communication channel. LoRa is not intended to replace the primary WebSocket connection during normal operation. Instead, it is activated when the primary communication channel becomes unavailable.

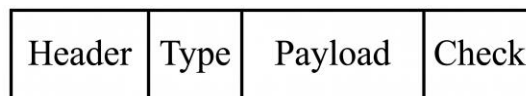
This provides the system with an alternative communication path that can be used to maintain essential communication during temporary network failures.

7.2 Lightweight Binary Packet Design

LoRa has significantly lower bandwidth compared with conventional Internet-based communication. Therefore, sending large or unnecessary messages over the backup channel would reduce communication efficiency.

The proposed communication protocol uses a lightweight binary packet structure containing only the information required for communication between the system components. This reduces packet size and avoids unnecessary overhead.

The packet structure can be represented conceptually as:



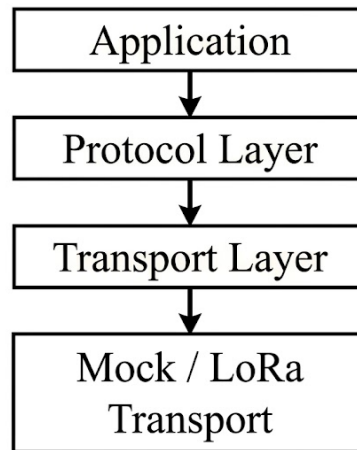
The exact packet fields can be adjusted according to the message type and the requirements of the final hardware implementation.

7.3 Transport and Protocol Architecture

The communication module separates the transport layer from the communication protocol. The protocol defines how messages are structured and interpreted, while the transport layer is responsible for transmitting and receiving the data.

This separation allows the current software implementation to use a mock transport during development and testing. A physical LoRa driver can later replace the mock transport without requiring major changes to the higher-level communication logic.

The architecture can therefore be summarized as:



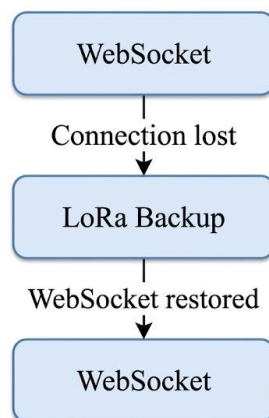
This design improves modularity and allows the communication system to be tested before the physical LoRa hardware is integrated.

7.4 Automatic Fallback Mechanism

The system monitors the actual state of the primary WebSocket connection. When the connection is operating normally, WebSocket remains the primary communication channel.

When the WebSocket connection becomes unavailable, the system automatically activates the backup communication path. When the primary connection is restored, the system can return to the normal communication channel.

The basic behavior is:

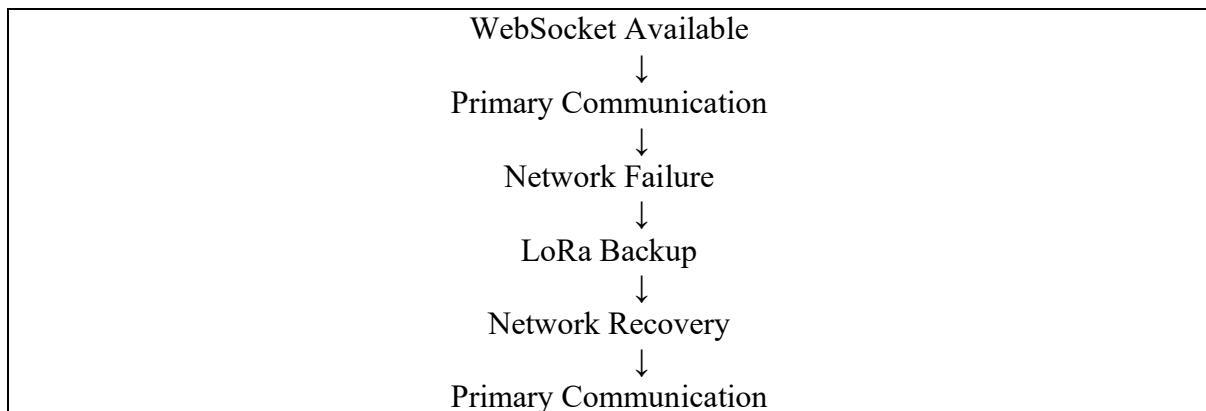


This mechanism allows the communication system to adapt to changes in network availability without requiring manual intervention.

7.5 Testing and Results

The fallback mechanism was evaluated by intentionally interrupting the primary network connection and monitoring the communication state. When the WebSocket connection was available, the system operated using the primary communication channel. After the connection was interrupted, the system detected the failure and changed to the backup communication path.

When the WebSocket connection was restored, the system detected the recovery and returned to the primary communication channel. The test therefore verified the intended two-way fallback behavior:



The test demonstrates that the fallback mechanism can respond automatically to changes in the primary communication state.

7.6 Limitations

The current implementation has not yet been validated using physical LoRa hardware. Therefore, practical characteristics such as actual communication range, radio interference, air-time latency, packet loss, and performance under real operating conditions have not yet been evaluated.

The current results should consequently be considered a software-level validation of the fallback mechanism and communication architecture. Physical LoRa testing will be required in the later thesis stage to evaluate the performance of the complete communication system.

CHAPTER 8: OBSTACLE DETECTION PROTOTYPE

8.1 Overview

Obstacle detection is introduced as an additional perception module to improve the UAV's awareness of its surrounding environment. The prototype combines YOLO-based object detection with Depth Anything V2 to obtain both semantic information and approximate depth information from camera images.

The YOLO model is responsible for detecting and classifying objects in the scene, while Depth Anything V2 estimates the relative depth of each pixel. By combining these two outputs, the system can identify potential obstacles and estimate their relative distance from the camera.

The current implementation is developed and evaluated as a standalone prototype. It has not yet been integrated into the main ROS 2 navigation pipeline or used as an active obstacle-avoidance controller.

8.2 YOLO-Based Object Detection

YOLO (You Only Look Once) is used to detect objects from camera frames. For each detected object, the model provides a bounding box, object class, and confidence score.

The detection process can be summarized as:

Camera Frame → YOLO Inference → Object Detection → Bounding Boxes and Classes

The detected objects provide semantic information about potential obstacles. For example, objects such as people, vehicles, or other structures can be identified and highlighted in the camera view.

8.3 Depth Estimation Using Depth Anything V2

While object detection provides information about what is present in the image, it does not directly provide the distance to the detected objects. Therefore, **Depth Anything V2** is used to estimate a depth map from the same camera frame.

The processing pipeline is:

Camera Frame → Depth Anything V2 → Depth Map → Relative Distance Information

The estimated depth can then be sampled inside a detected object's bounding box to obtain an approximate relative distance. This allows the system to distinguish between objects that are closer to or farther from the UAV.

8.4 Combined Perception Pipeline

The two models are combined to provide both object identity and depth information. YOLO first identifies potential objects, while the depth model provides corresponding spatial information.

The overall prototype pipeline is:

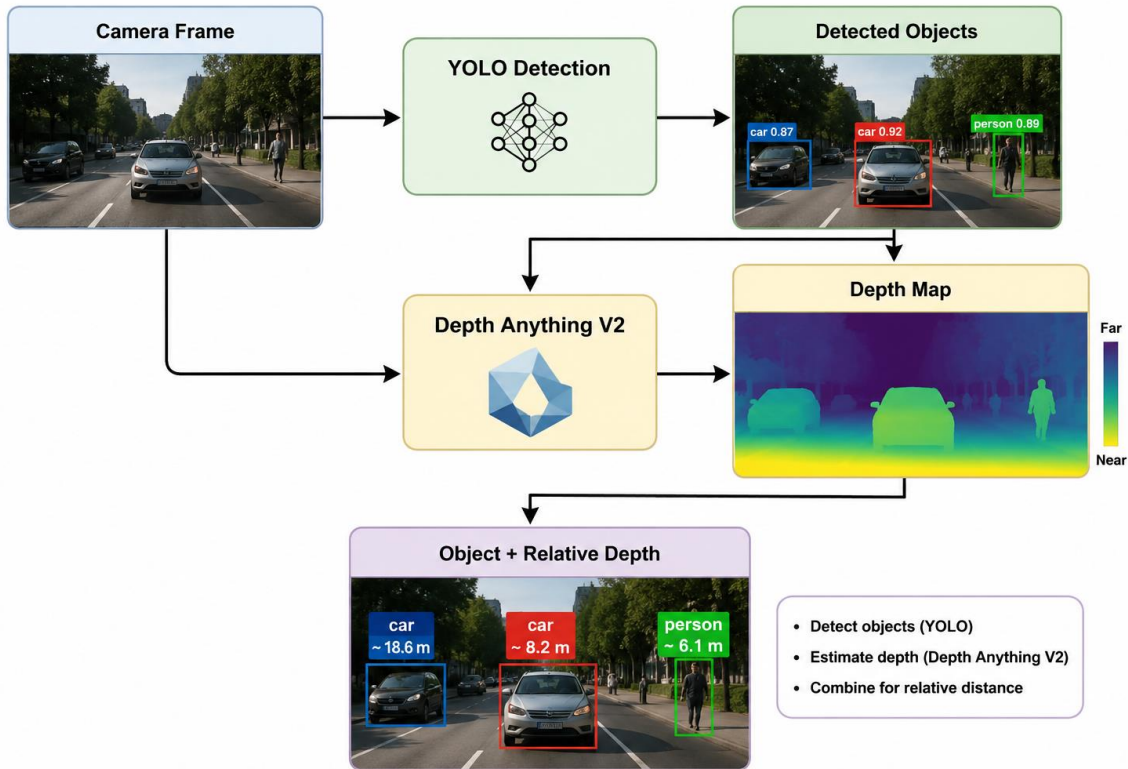


Figure 8.1: YOLO and Depth Anything V2-Based Obstacle Detection Pipeline

This combination provides a foundation for future obstacle-aware navigation. However, the current prototype only performs perception and does not directly command the UAV to avoid detected obstacles.

8.5 Prototype Evaluation

The prototype is evaluated using camera images to verify whether objects can be detected correctly and whether corresponding depth information can be obtained. The evaluation focuses on the consistency between detected objects and their estimated relative depth.

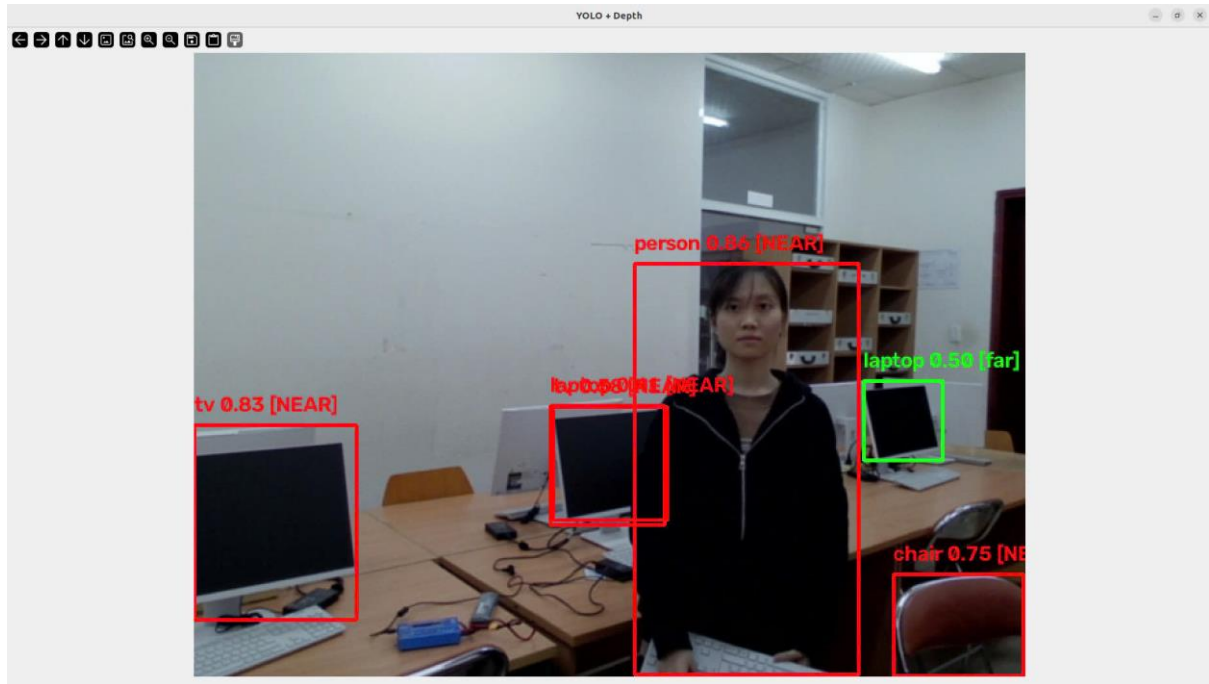


Figure 8.2: YOLO-Based Object Detection with Relative Depth Estimation

The results demonstrate the feasibility of combining object detection and monocular depth estimation for UAV perception. However, quantitative evaluation such as obstacle-distance error, detection accuracy, and real-time processing performance should be performed in future experiments.

8.6 Limitations

The current obstacle detection system remains a prototype and has not yet been integrated into the official ROS 2 UAV navigation architecture. In addition, monocular depth estimation provides relative depth rather than directly measured metric distance, so the estimated depth should not be considered an accurate physical distance without further calibration.

Another limitation is that the prototype has not been validated extensively under real UAV flight conditions. Factors such as lighting changes, motion blur, camera vibration, and computational constraints may affect detection and depth-estimation performance.

CHAPTER 9: SYSTEM INTEGRATION AND TESTING

9.1 Testing Methodology

The UAV system consists of several independent modules, including communication, localization, precision landing, and perception. During the current development stage, the system is tested using separate scenarios rather than executing all modules simultaneously. This approach allows each function to be evaluated independently and makes it easier to identify the source of errors or unexpected behavior.

Three main testing scenarios are considered: precision landing from the ground dashboard to the UAV, GPS-to-VIO localization switching in SITL, and automatic communication fallback when the primary network connection is interrupted.

9.2 Scenario A – Precision Landing End-to-End

The first scenario evaluates the complete end-to-end precision-landing workflow, starting from the emergency request at the ground dashboard and ending with the successful landing of the UAV. The test begins when the user sends an SOS request through the ground dashboard. The request is then received by the dispatch dashboard, where the operator can dispatch the UAV to the reported location.

The overall workflow is:

Send SOS → Dispatch Drone → UAV Flight → Target Area → ArUco Detection → Alignment → Descent → AUTO.LAND → Touchdown

The first step is the initiation of the emergency request. The user presses the SEND SOS button on the ground dashboard, while the dispatch dashboard receives the request and provides the option to dispatch the UAV.

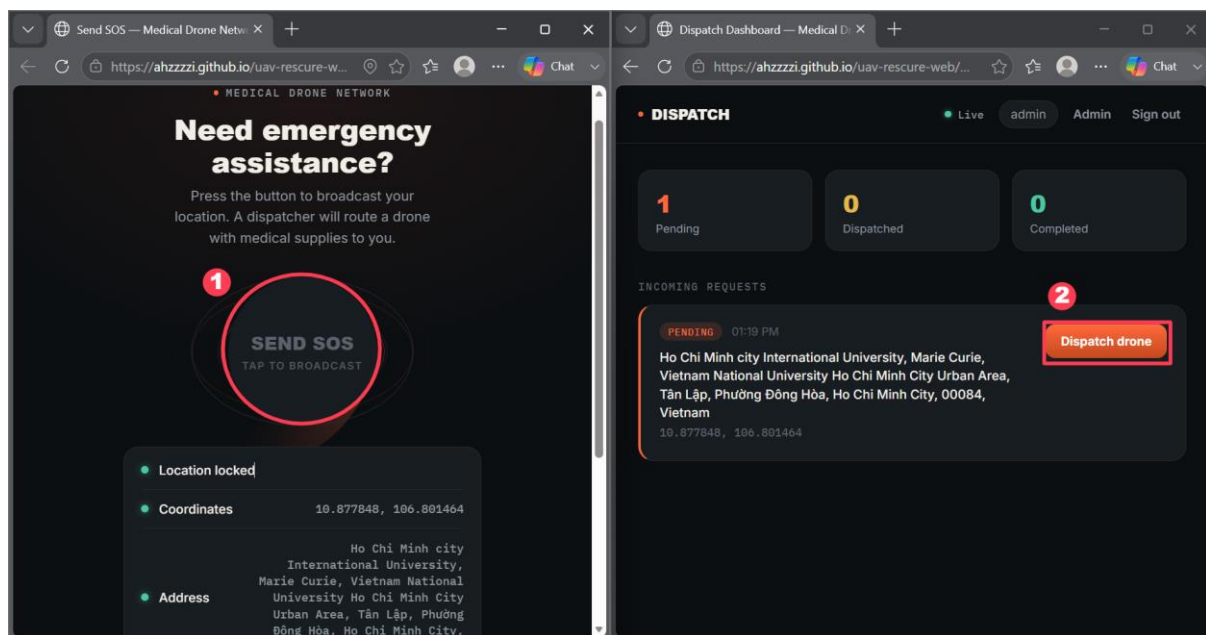


Figure 9.1: Emergency Request and Drone Dispatch Interface

After the request is dispatched, the UAV begins its flight toward the target location. The flight controller receives the mission information through the communication bridge and executes the navigation process toward the target area.

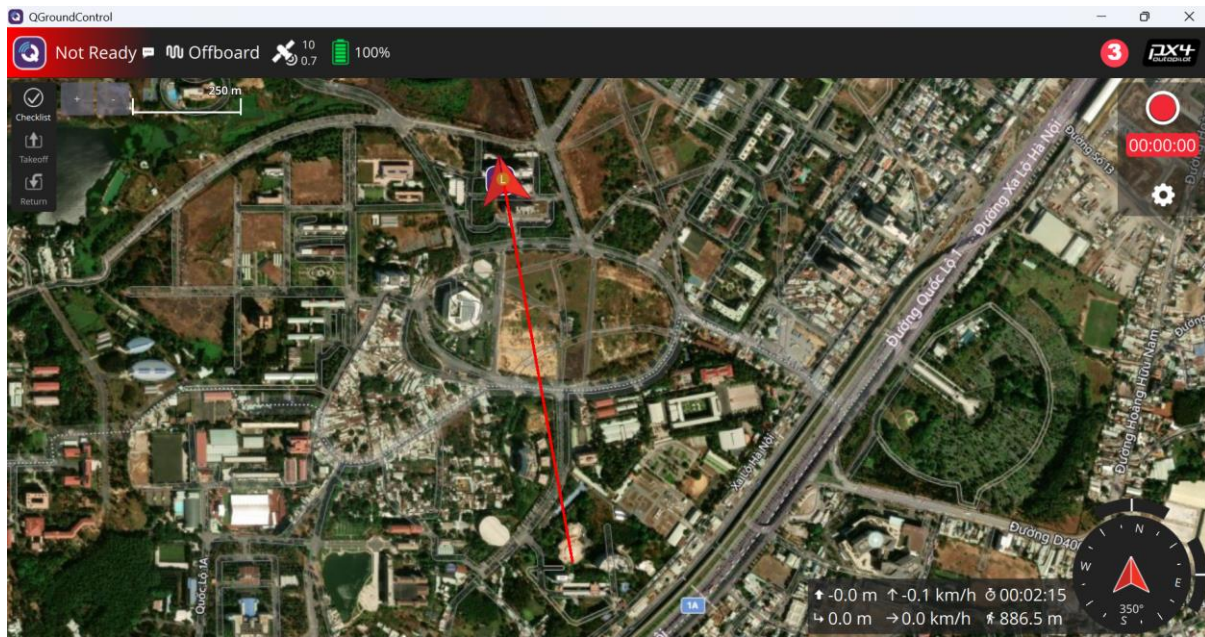


Figure 9.2: UAV Navigation from the Initial Position to the Target Area in QGroundControl

When the UAV reaches the target area, the precision-landing module activates the camera-based landing process. The camera searches for the ArUco marker placed at the landing location. Once the marker is detected, the system estimates its relative position and begins aligning the UAV with the landing target.

```
huyen@HUYEN:/mnt/c/Users/trant$ ros2 run precision_landing landing_controller_node
[INFO] [1788416559.064734462] [landing_controller_node]: Landing Controller starting...
[INFO] [1788416559.127967060] [landing_controller_node]: Landing Controller ready. Waiting for activation...
[INFO] [1788416766.438415710] [landing_controller_node]: Activated by bridge node. Starting search for marker.
[INFO] [1788416767.084642269] [landing_controller_node]: Marker acquired. Switching to ALIGNING.
[INFO] [1788416767.147013016] [landing_controller_node]: Aligned over marker. Switching to DESCENDING.
[INFO] [1788416795.128279076] [landing_controller_node]: Close enough to marker (Z < threshold). Handing off to FCU AUTO.LAND - avoid
ing OFFBOARD near ground to prevent ground-effect turbulence.
[INFO] [1788416795.240697532] [landing_controller_node]: AUTO.LAND mode accepted by PX4.
```

Figure 9.3: Precision Landing Controller Execution and PX4 AUTO.LAND Handoff

After achieving sufficient alignment, the UAV enters the descent stage. The landing controller continuously processes the marker position while commanding the UAV to descend toward the landing point.

```

[INFO] [1788416794.128865977] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.228791737] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.329240974] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.432285907] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.529530790] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.629822054] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.729487522] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.831448644] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416794.930637305] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416795.030333232] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416795.130066328] [uav_cloud_bridge]: Precision landing status: DESCENDING
[INFO] [1788416795.229663548] [uav_cloud_bridge]: Precision landing status: AUTO_LANDING
[INFO] [1788416795.243777573] [uav_cloud_bridge]: Precision landing status: LANDED
[INFO] [1788416795.331998537] [uav_cloud_bridge]: Precision landing status: LANDED
[INFO] [1788416795.627853880] [uav_cloud_bridge]: Precision landing succeeded.
[INFO] [1788416795.632643632] [uav_cloud_bridge]: Mission timeline finished.

```

Figure 9.4: Precision Landing Status Transition and Successful Landing

When the UAV is sufficiently close to the ground, the landing controller hands the final landing operation to PX4 through the AUTO.LAND mode. The UAV then completes the touchdown sequence.

```

INFO [commander] Ready for takeoff!
INFO [ekf2] 0 - New NED origin (LLA): 10.8511680000, 106.6494860000, 10.000
INFO [px4] Startup script returned successfully
pxh> WARN [commander] command 520 unsupported
INFO [tone_alarm] notify negative
WARN [commander] command 520 unsupported
INFO [tone_alarm] notify negative
WARN [commander] command 520 unsupported
INFO [tone_alarm] notify negative
WARN [commander] command 520 unsupported
INFO [tone_alarm] notify negative
WARN [commander] command 520 unsupported
INFO [tone_alarm] notify negative
WARN [commander] command 520 unsupported
INFO [tone_alarm] notify negative
INFO [commander] Armed by external command
INFO [tone_alarm] arming warning
INFO [commander] Takeoff detected
INFO [commander] Landing detected
INFO [commander] Disarmed by landing
INFO [tone_alarm] notify neutral
INFO [vehicle_imu] ACC 0 (1310988) offset committed: [-0.008 -0.050 -0.023]->[-0.039 -0.108 -0.028])
INFO [logger] closed logfile, bytes written: 35266888
NodeShared::Publish() Error: Interrupted system call

```

Figure 9.5: PX4 Flight Controller Log Confirming Takeoff, Landing, and Disarming

The final dashboard status is then updated to indicate that the delivery/mission has been completed.

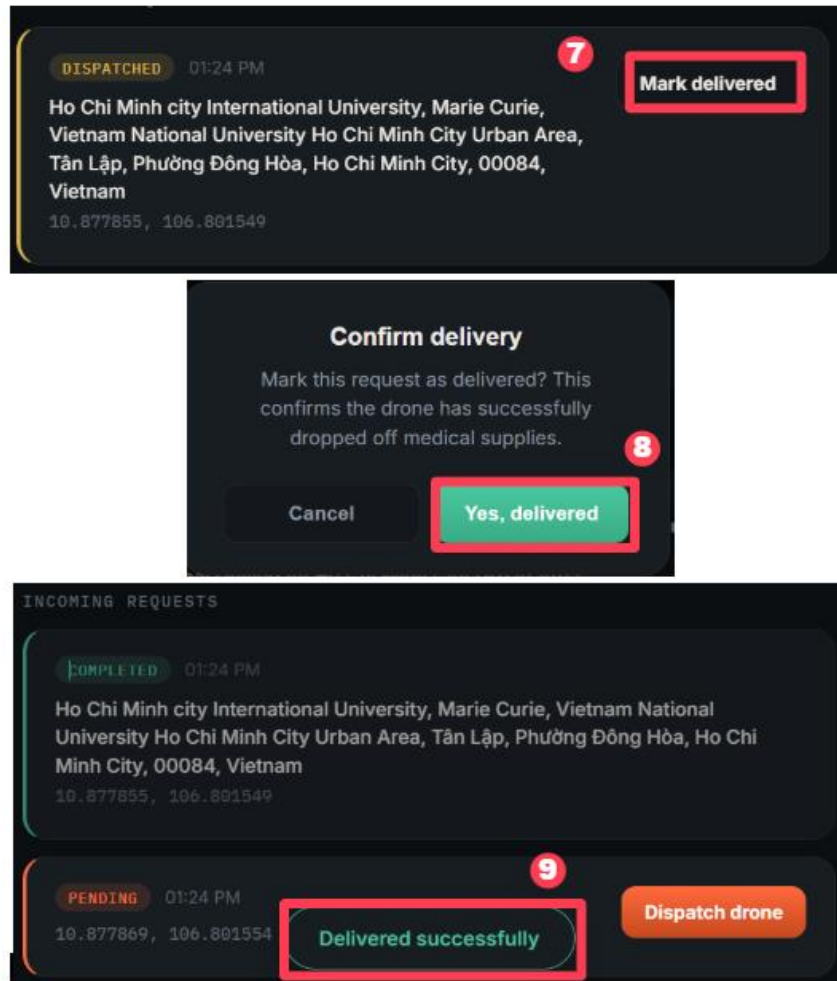


Figure 9.6: Completed Emergency Request

Overall, this scenario verifies the interaction between the ground-control dashboard, communication bridge, PX4 flight controller, camera-based perception, and precision-landing controller. The test is considered successful when the emergency request is transmitted correctly, the UAV reaches the target area, the ArUco-based landing sequence is executed, and the UAV successfully completes the landing process.

9.3 Scenario B – GPS-to-VIO Switching in SITL

The second scenario evaluates the adaptive localization mechanism in the PX4 Software-In-The-Loop environment. The test verifies whether the system can switch from GPS to VIO when GPS is considered unreliable and subsequently return to GPS when GPS becomes available again.

The test sequence is:

Normal GPS Operation → GPS Unavailable → Switch to VIO → VIO-Based Operation → GPS Recovery → Return to GPS

The switching logic evaluates the validity of both localization sources before selecting the active source. This scenario is used to verify the two-way transition behavior described in Chapter 5.

```

[INFO] [1788417876.827123789] [gps_vio_switching_node]: GPS-to-VIO Switching Node started (MAVROS State-Aware Mode)
[INFO] [1788417877.262976191] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: GPS
[INFO] [1788417877.753439825] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: GPS
[WARN] [1788417877.756492292] [gps_vio_switching_node]: SWITCHING LOCALIZATION SOURCE: GPS -> VIO
[INFO] [1788417878.254179977] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417878.752762832] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417879.252437445] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417879.752302325] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417880.252151350] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417880.752219245] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417881.252441333] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417881.752098727] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417882.252006178] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417882.753603164] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417883.254142978] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417883.753149647] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417884.255006077] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417884.753899065] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417885.253682209] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417885.753029921] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417886.253019280] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417886.751983653] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417887.252493964] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417887.752060476] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417888.251782447] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417888.754117328] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417889.260895621] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417889.753628186] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417890.252937153] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417890.753631864] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417891.255388759] [gps_vio_switching_node]: STATUS -> gps_ok: False, vio_ok: True, Active Source: VIO
[INFO] [1788417891.756123418] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: VIO
[WARN] [1788417891.758801101] [gps_vio_switching_node]: SWITCHING LOCALIZATION SOURCE: VIO -> GPS
[INFO] [1788417892.252923906] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
[INFO] [1788417892.752300325] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
[INFO] [1788417893.255312348] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS
[INFO] [1788417893.753630138] [gps_vio_switching_node]: STATUS -> gps_ok: True, vio_ok: True, Active Source: GPS

```

Figure 9.7: GPS-to-VIO-to-GPS Localization Source Switching

9.4 Scenario C – Automatic LoRa Fallback

The third scenario evaluates the automatic fallback mechanism from the primary WebSocket communication channel to the LoRa backup transport. The test verifies that the UAV Cloud Bridge can detect a loss of the primary network connection, activate the LoRa fallback transport, and recover the primary connection when network connectivity is restored.

The initial system configuration is shown below. The UAV Cloud Bridge is connected to the relay server, while the LoRa ground-station mock and LoRa protocol node are running with a mock transport. This setup represents the normal communication state before a network failure occurs.

```

huyen@HUYEN:~/drone_project/ros2_ws$ ros2 run precision_landing lora_ground_station_mock
[INFO] [1788500574.607917225] [lora_ground_station_mock]: Mock LoRa Ground Station listening...

huyen@HUYEN:~/mnt/c/Users/trant$ ros2 run uav_cloud_bridge bridge_node
[INFO] [1788500590.277645204] [uav_cloud_bridge]: UAV Cloud Bridge starting...
[INFO] [1788500590.338549121] [uav_cloud_bridge]: Bridge node ready.
[INFO] [1788500590.714709321] [uav_cloud_bridge]: Connected to relay server.

^Chuyen@HUYEN:~/mnt/c/Users/trant$ ros2 run precision_landing lora_protocol_node
[INFO] [1788500582.825316589] [lora_protocol_node]: LoRa Protocol Node started (mock transport, drone role)

```

Figure 9.8: Detection of Primary Network Failure and Activation of LoRa Fallback

The primary network connection is then intentionally interrupted by turning off the Wi-Fi connection. The UAV Cloud Bridge detects that the WebSocket connection has been lost through the keepalive mechanism and reports a primary network connection error. The bridge subsequently switches to the LoRa fallback transport and attempts to maintain the backup communication path.

```

huyen@HUYEN:/mnt/c/Users/trant$ ros2 run uav_cloud_bridge bridge_node
[INFO] [1788500590.277645204] [uav_cloud_bridge]: UAV Cloud Bridge starting...
[INFO] [1788500590.338549121] [uav_cloud_bridge]: Bridge node ready.
[INFO] [1788500590.714709321] [uav_cloud_bridge]: Connected to relay server.
[ERROR] [1788500848.116838111] [uav_cloud_bridge]: Primary network connection lost: sent 1011 (internal error) keepalive ping timeout
; no close frame received
[WARN] [1788500848.117768065] [uav_cloud_bridge]: Switching to LoRa Fallback Transport... Reconnecting in 5s
[ERROR] [1788500848.118751789] [uav_cloud_bridge]: Primary network connection lost: sent 1011 (internal error) keepalive ping timeout
; no close frame received
[WARN] [1788500848.119655976] [uav_cloud_bridge]: Switching to LoRa Fallback Transport...
[ERROR] [1788500848.120572833] [uav_cloud_bridge]: Primary network connection lost: sent 1011 (internal error) keepalive ping timeout
; no close frame received
[WARN] [1788500848.121937738] [uav_cloud_bridge]: Switching to LoRa Fallback Transport...
[ERROR] [1788500848.123719174] [uav_cloud_bridge]: Primary network connection lost: sent 1011 (internal error) keepalive ping timeout
; no close frame received
[WARN] [1788500848.124351849] [uav_cloud_bridge]: Switching to LoRa Fallback Transport...
[ERROR] [1788500848.124841742] [uav_cloud_bridge]: Primary network connection lost: sent 1011 (internal error) keepalive ping timeout
; no close frame received
[WARN] [1788500848.125402972] [uav_cloud_bridge]: Switching to LoRa Fallback Transport...
[ERROR] [1788500848.125873060] [uav_cloud_bridge]: Primary network connection lost: sent 1011 (internal error) keepalive ping timeout
; no close frame received
[WARN] [1788500848.126374771] [uav_cloud_bridge]: Switching to LoRa Fallback Transport...

```

Figure 9.9: Detection of Primary Network Failure and Switching to LoRa Fallback

After the network connection is restored, the UAV Cloud Bridge attempts to reconnect to the relay server. The terminal output confirms that the connection is successfully re-established, demonstrating the recovery of the primary communication channel after the fallback condition.

```

[WARN] [1788500848.126374771] [uav_cloud_bridge]: Switching to LoRa Fallback Transport...
[ERROR] [1788500863.132022281] [uav_cloud_bridge]: Primary network connection lost: timed out during opening handshake
[WARN] [1788500863.132659914] [uav_cloud_bridge]: Switching to LoRa Fallback Transport... Reconnecting in 5s
[ERROR] [1788500878.147539759] [uav_cloud_bridge]: Primary network connection lost: timed out during opening handshake
[WARN] [1788500878.148197961] [uav_cloud_bridge]: Switching to LoRa Fallback Transport... Reconnecting in 5s
[INFO] [1788500883.520136819] [uav_cloud_bridge]: Connected to relay server.

```

Figure 9.10: Primary Network Reconnection After LoRa Fallback

Overall, this scenario demonstrates the intended automatic communication-fallback behavior. The system successfully detects the loss of the primary WebSocket connection, activates the LoRa fallback transport, and reconnects to the primary relay server after network connectivity is restored. Since the LoRa component is implemented using a mock transport in this test, the results validate the software fallback mechanism and communication architecture, rather than the physical performance of a LoRa radio link.

9.5 Testing Environment and Tools

The testing environment combines simulation and real hardware components according to the requirements of each module. PX4 SITL and Gazebo headless are used for UAV simulation, while MAVROS2 provides the communication interface between the ROS 2 software components and PX4. A real webcam is used for camera-based perception and precision-landing experiments.

The main testing components are:

- **PX4 SITL** – UAV flight-control simulation.
- **Gazebo headless** – simulated UAV environment.
- **MAVROS2** – ROS 2 to PX4 communication interface.
- **Real webcam** – camera input for ArUco detection and obstacle-detection experiments.
- **Ground dashboard and bridge node** – command and communication testing.

This testing setup allows the major system functions to be validated individually while keeping the testing environment manageable during the current development stage.

CHAPTER 10: LIMITATIONS AND DISCUSSION

10.1 Development Environment Limitations

The current development environment is based on WSL2, which provides a convenient Linux environment for developing and testing the ROS 2 and PX4 software stack on a Windows host. However, WSL2 introduces limitations for long-running real-time simulation, particularly in terms of timing consistency and hardware-level access.

These limitations may affect the stability of extended SITL experiments and can introduce differences between simulation behavior and the behavior expected on a dedicated Linux system or embedded onboard computer.

10.2 Limitations of Simulation-Based Testing

Most UAV functions in the current implementation are validated using simulation or software-level testing. PX4 SITL provides a controlled environment for testing localization switching and flight-control logic, but it cannot fully reproduce the conditions of a physical UAV.

In particular, the current system has not yet been evaluated with real GPS degradation, real sensor noise, physical UAV vibration, or real flight dynamics. Similarly, the precision-landing module has been tested using a real webcam together with a simulated UAV, which does not completely reproduce the camera motion and physical relationship between the UAV and the landing marker.

Therefore, the current results demonstrate the functional feasibility of the implemented algorithms, but further hardware-based testing is required before evaluating the system under real operational conditions.

10.3 Technical Limitations of Individual Modules

Each module has specific limitations that should be considered.

The GPS/VIO switching module has been validated in SITL, but its robustness under real GPS multipath, signal degradation, and VIO tracking failure has not yet been fully evaluated.

The precision-landing module depends on reliable ArUco marker detection. Changes in lighting, motion blur, camera vibration, viewing angle, and temporary marker occlusion can affect detection performance. The current implementation also relies on a grace-period mechanism when the marker is temporarily lost.

The LoRa communication module has been tested at the software level using a backup transport mechanism, but physical LoRa hardware has not yet been evaluated. Therefore, actual communication range, RF interference, air-time latency, and packet-loss behavior remain to be measured.

The obstacle-detection prototype, based on YOLO and Depth Anything V2, has demonstrated object detection and relative depth estimation, but it has not yet been integrated into the main ROS 2 navigation system. Its depth output also represents relative depth rather than directly calibrated metric distance.

10.4 High-Level Control Arbitration

Another limitation is the absence of a dedicated high-level control arbitration mechanism for situations in which multiple nodes may attempt to influence the UAV simultaneously.

For example, the localization module, precision-landing controller, obstacle-detection module, and other navigation components may potentially generate different control requirements. Without a centralized arbitration layer, conflicts between these commands could occur when multiple modules become active at the same time.

A future version of the system should therefore introduce a high-level controller or arbitration layer that determines which module has control authority based on the current mission state and safety conditions. This would improve coordination between navigation, landing, obstacle avoidance, and emergency behaviors.

CHAPTER 11: FUTURE WORK AND THESIS PLAN

11.1 Hardware Deployment

The current pre-thesis implementation has been developed primarily in a software and simulation-based environment. Therefore, the first major objective of the official thesis phase is to transfer the developed system to the actual onboard computing platform. The Jetson Nano will be prepared with the appropriate JetPack environment, after which the ROS 2 workspace and the required software dependencies will be rebuilt for the ARM-based architecture.

Particular attention will be given to the computational requirements of the perception and localization modules. ORB-SLAM3, YOLO, and Depth Anything V2 may require significant processing resources, especially when multiple modules operate simultaneously. The thesis phase will therefore evaluate processing time, CPU/GPU utilization, memory consumption, and overall system responsiveness on the Jetson Nano.

The final objective of this stage is to establish a stable onboard software environment before connecting the system to the physical UAV.

11.2 Real Camera Calibration and Integration

The camera used during the pre-thesis experiments will be replaced or supplemented by the actual CSI camera intended for deployment on the UAV. A complete camera calibration procedure will be performed to obtain the intrinsic parameters required by the visual perception and localization modules.

The calibration results will be used by both the VIO and precision-landing systems. This is particularly important because inaccurate camera parameters can introduce errors in pose estimation and marker-based distance estimation.

In addition, a wide-angle or fisheye camera will be evaluated if its field of view provides advantages for the intended UAV application. The selected camera configuration will be tested under different viewpoints and lighting conditions to determine its suitability for navigation and landing.

11.3 Integration with Real Flight Hardware

After preparing the onboard computer and camera system, the next stage will integrate the software with the physical flight-control hardware. The Pixhawk flight controller will be connected to the Jetson Nano and configured to communicate with the companion computer through the SIYI Air Unit.

The SIYI MK15 remote controller will also be integrated into the system. Manual override functionality will be configured so that the operator can take control of the UAV whenever the autonomous system behaves unexpectedly or an unsafe situation occurs.

Initial hardware tests will be performed without autonomous flight. Communication between the Jetson Nano, Pixhawk, Air Unit, and remote controller will first be verified independently before gradually introducing autonomous functions.

11.4 Real-World Flight Testing

Once the hardware integration is stable, real flight experiments will be conducted to validate the modules developed during the pre-thesis stage. The testing will gradually progress from basic manual flights to controlled autonomous experiments.

The GPS/VIO switching mechanism will be evaluated under conditions where GPS quality changes during flight. The objective is to determine whether the switching logic can maintain stable localization when GPS becomes unreliable and safely return to GPS when reliable measurements are restored.

The precision-landing module will then be evaluated using a physical landing marker. Unlike the current simulation-based experiments, the marker and camera will move according to the actual UAV motion. This will allow the effects of vibration, motion blur, lighting, and camera perspective to be evaluated.

The backup communication system and obstacle-detection module will also be tested during real-world operation where possible. The results will be compared with the current SITL and software-level experiments to identify differences between simulated and physical environments.

11.5 Physical LoRa Implementation

The current YOLO and Depth Anything V2 implementation remains at the prototype level. In the thesis phase, the perception pipeline will be converted into a dedicated ROS 2 node so that obstacle information can be published and consumed by other components of the UAV system.

The ROS 2 node will process camera frames, perform object detection, estimate relative depth, and publish the resulting obstacle information. The interface between the perception node and the navigation system will then be defined so that detected obstacles can be used during flight.

Initially, the module may provide obstacle warnings without directly controlling the UAV. After the perception output has been validated, it can be extended to support obstacle-aware navigation or avoidance behavior.

11.6 High-Level Control Arbitration

One of the remaining architectural tasks is the development of a high-level control arbitration mechanism. The current system contains several independent modules that may potentially require control authority, including normal navigation, precision landing, obstacle detection, emergency behavior, and manual override.

During the thesis phase, a centralized mechanism will be developed to determine which module has control authority based on the current mission state and safety conditions. For example, precision landing should have control priority during the landing phase, while manual override should be able to take control when required for safety.

This arbitration layer will provide a clearer hierarchy between the different autonomous functions and reduce the possibility of conflicting commands being sent to the flight controller.

Overall, the thesis phase will extend the current pre-thesis work from software-level and simulation validation toward physical UAV implementation and real-world testing. The final

goal is to evaluate whether the proposed architecture can operate reliably as an integrated autonomous emergency medical delivery system under realistic operating conditions.

CHAPTER 12: CONCLUSION

12.1 Summary of Achieved Results

This pre-thesis presented the design and software-level validation of an autonomous UAV system for emergency medical delivery in environments where GPS availability and communication reliability may be limited. The proposed system was developed using a ROS 2-based architecture and was evaluated primarily through PX4 Software-In-The-Loop (SITL), together with real camera and perception experiments.

Several core functions were successfully implemented and evaluated. The VIO/SLAM module provides visual localization for GPS-denied operation, while the adaptive GPS/VIO switching mechanism selects the appropriate localization source based on the validity of the available data. The precision-landing module uses ArUco marker detection and pose estimation to guide the UAV during the final landing phase and includes safety handling for temporary marker loss. In addition, a LoRa-based backup communication architecture was developed to provide an alternative communication path when the primary WebSocket connection becomes unavailable.

An additional obstacle-detection prototype was also developed by combining YOLO object detection with Depth Anything V2 depth estimation. Although this module has not yet been integrated into the main ROS 2 navigation pipeline, the initial results demonstrate the feasibility of combining semantic object detection with relative depth information for future obstacle-aware navigation.

12.2 Conclusion and Feasibility

The results obtained during the pre-thesis stage demonstrate that the proposed software architecture is technically feasible for further development. The individual modules were successfully implemented and tested in their respective environments, while the integration tests demonstrated important system behaviors such as precision-landing execution, GPS/VIO localization switching, and communication fallback.

However, the current work remains primarily at the simulation and prototype stage. The system has not yet been validated through real UAV flight experiments, and several components still require physical hardware testing. Therefore, the results should be considered as a foundation for the official thesis rather than a complete validation of the final UAV system.

The next phase will focus on deploying the software to the Jetson Nano, integrating the real camera and flight hardware, implementing physical LoRa communication, converting the obstacle-detection prototype into a ROS 2 node, and conducting real-world flight experiments. These steps will allow the proposed architecture to be evaluated under realistic operating conditions and provide the basis for completing the autonomous emergency medical delivery system.

REFERENCES

- Campos, C., Elvira, R., Rodríguez, J. J. G., Montiel, J. M. M., & Tardós, J. D. (2021). ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6), 1874–1890. <https://doi.org/10.1109/TRO.2021.3075644>
- Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., & Marín-Jiménez, M. J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6), 2280–2292. <https://doi.org/10.1016/j.patcog.2014.01.005>
- Open Robotics. (n.d.). *ROS 2 documentation*. <https://docs.ros.org/>
- Open Source Robotics Foundation. (n.d.). *MAVROS*. <https://github.com/mavlink/mavros>
- OpenCV. (n.d.). *OpenCV documentation*. <https://docs.opencv.org/>
- PX4 Autopilot. (n.d.). *PX4 user guide*. <https://docs.px4.io/>
- Semtech. (n.d.). *LoRa and LoRaWAN*. <https://www.semtech.com/lora>
- Ultralytics. (n.d.). *Ultralytics YOLO documentation*. <https://docs.ultralytics.com/>
- Yang, L., Kang, B., Huang, Z., Zhao, Z., Xu, X., Feng, J., & Zhao, H. (2024). Depth Anything V2. *Advances in Neural Information Processing Systems*, 37.